MỞ ĐẦU

1. Lí do chọn đề tài

Du lịch homestay tại Đồng bằng sông Cửu Long nói chung và Trà Vinh nói riêng đang tăng nhanh cùng làn sóng du lịch trải nghiệm và du lịch cộng đồng. Khác với khách sạn, homestay thường do hộ gia đình tự vận hành, quy mô từ vài phòng tới vài chục phòng, và không có bộ phận lễ tân trực suốt ngày đêm.

Thực tế vận hành của nhóm cơ sở lưu trú này bộc lộ ba vấn đề.

Thứ nhất, kênh đặt phòng phân mảnh. Chủ homestay nhận đặt phòng qua nhiều kênh tin nhắn khác nhau và đôi khi qua sàn trung gian. Mỗi kênh là một danh sách riêng trong trí nhớ hoặc trên một cuốn sổ. Không có nơi nào cho biết một ngày cụ thể còn bao nhiêu phòng ngoài việc tự nhớ lại.

Thứ hai, rủi ro đặt trùng phòng là rủi ro thường trực. Khi hai khách liên hệ gần như cùng lúc và chủ nhà trả lời còn phòng cho cả hai, sai sót chỉ lộ ra vào ngày nhận phòng, lúc không còn cách khắc phục nào ngoài việc từ chối một khách đã đi đường xa tới nơi. Đây không phải lỗi bất cẩn mà là hệ quả tất yếu của việc quản lý lịch phòng bằng trí nhớ con người.

Thứ ba, chi phí hoa hồng của sàn trung gian. Các sàn đặt phòng trực tuyến thu khoảng 15–20% giá trị mỗi đơn. Với homestay quy mô nhỏ, đó là phần lớn biên lợi nhuận. Nhưng rời sàn thì mất luôn kênh tiếp cận khách.

Một website đặt phòng riêng giải quyết cả ba vấn đề: gom mọi đơn về một nơi, để cơ sở dữ liệu chứ không phải trí nhớ con người bảo đảm không bán trùng phòng, và giữ lại toàn bộ doanh thu.

Về mặt học thuật, đề tài có giá trị riêng. Bài toán chống đặt trùng phòng là một bài toán tranh chấp dữ liệu kinh điển, không giải được đúng bằng cách kiểm tra thông thường ở tầng ứng dụng. Nó đòi hỏi hiểu biết về giao dịch cơ sở dữ liệu, mức cô lập và ràng buộc toàn vẹn ở tầng lưu trữ — những nội dung cốt lõi của ngành mà một đề tài quản lý thông thường không chạm tới.

2. Mục đích nghiên cứu

Xây dựng một website hoạt động được, có dữ liệu minh hoạ, giải quyết đúng bài toán giới thiệu và đặt phòng homestay, gồm ba khối: trang bán hàng cho khách, khu quản trị cho chủ homestay, và giao diện lập trình ứng dụng kết nối hai khối.

Tám mục đích cụ thể cùng tiêu chí nghiệm thu tương ứng:

Bảng 0.1 — *Bốn mục đích nghiên cứu và tiêu chí nghiệm thu tương ứng (Nguồn: tác giả)*

#	Mục đích	Tiêu chí nghiệm thu
1	Khách tìm được phòng trống theo ngày và đặt phòng thành công	Đặt được đơn từ trang chủ tới khi có mã QR
2	Hệ thống không bao giờ bán trùng một phòng	Kiểm thử đa luồng: đúng một luồng thắng
3	Đặt cọc qua mã QR, webhook tự xác nhận	Đơn tự chuyển trạng thái khi tiền về đủ
4	Không có nhánh nào để tiền của khách biến mất im lặng	Mọi khoản không khớp vào hàng đợi đối soát
5	Quản trị viên quản lý được phòng, giá, số khách và ngày khả dụng	Đóng phòng theo khoảng ngày, hệ thống tự mở lại
6	Giao diện đạt chuẩn trang đặt phòng thương mại	Không còn lỗi vi phạm WCAG mức A và AA
7	Một lệnh là đủ để chạy toàn hệ thống kèm dữ liệu mẫu	Ba lệnh từ lúc sao chép mã nguồn tới lúc chạy
8	Tài liệu kỹ thuật đầy đủ	12 tài liệu kỹ thuật kèm mã nguồn

3. Đối tượng nghiên cứu

Đề tài nghiên cứu ba đối tượng:

1. **Quy trình nghiệp vụ** đặt phòng lưu trú ngắn ngày của homestay quy mô nhỏ, từ lúc khách tìm phòng tới lúc trả phòng.
2. **Cơ chế bảo đảm toàn vẹn dữ liệu khi có tranh chấp** trong PostgreSQL: kiểu dữ liệu khoảng, ràng buộc loại trừ và chỉ mục GiST.
3. **Kiến trúc ứng dụng web tách rời** giữa tầng giao diện và tầng máy chủ, giao tiếp qua giao diện lập trình ứng dụng REST.

4. Phạm vi nghiên cứu

Về nghiệp vụ: từ lúc khách tìm phòng tới lúc trả phòng, cộng với đối soát thanh toán và báo cáo doanh thu. Không bao gồm kế toán, thuế, nhân sự, kho.

Về kỹ thuật: Java 21 và Spring Boot 3.5.6 ở tầng máy chủ; Angular 21 và Tailwind CSS 4 ở tầng giao diện; PostgreSQL 16 làm cơ sở dữ liệu; Docker Compose để đóng gói.

Về dữ liệu: dữ liệu minh hoạ tự sinh, mô phỏng một homestay 15 phòng thuộc 4 loại phòng.

Ngoài phạm vi. Việc nêu rõ giới hạn là một phần của thiết kế:

Bảng 0.2 — *Những nội dung không thực hiện trong phạm vi đồ án và lý do (Nguồn: tác giả)*

Không thực hiện	**Lý do**
Kết nối cổng thanh toán thật	Đề tài không yêu cầu; hệ thống viết đúng luồng thật và đọc khoá từ biến môi trường
Hoàn tiền tự động	Hoàn tiền là quyết định kinh doanh; hệ thống đưa vào hàng đợi cho người xử lý
Ứng dụng di động	Website đã đáp ứng trên điện thoại
Đa ngôn ngữ và đa chi nhánh	Không thuộc lõi nghiệp vụ của một cơ sở đơn lẻ
Giao thức bảo mật HTTPS trong bản đóng gói	Cần tên miền và chứng chỉ thật; là giới hạn đã biết, trình bày ở mục 4.6

CHƯƠNG 1. TỔNG QUAN

1.1. Giới thiệu bài toán

1.1.1. Mô tả cơ sở

Homestay TVH là một cơ sở lưu trú đặt tại Trà Vinh, quy mô **15 phòng vật lý** thuộc **4 loại phòng**: Standard, Deluxe, Family và Bungalow. Mỗi loại phòng có giá cơ bản theo đêm, sức chứa người lớn và trẻ em riêng, bộ tiện nghi riêng và bộ ảnh riêng.

Nghiệp vụ cần tin học hoá gồm bốn nhóm:

Nhóm 1 — Giới thiệu. Trình bày loại phòng, tiện nghi, hình ảnh, tin tức và đánh giá của khách cũ, sao cho khách chưa từng biết tới homestay vẫn hình dung được họ sẽ nhận được gì.

Nhóm 2 — Đặt phòng. Khách chọn khoảng ngày và số khách, hệ thống trả về các loại phòng còn chỗ kèm chi phí dự kiến, khách gửi yêu cầu đặt phòng và đặt cọc.

Nhóm 3 — Quản lý vận hành. Chủ homestay theo dõi đơn theo trạng thái, xác nhận hoặc huỷ, đối soát tiền đã nhận, và quản lý danh mục phòng, giá, khuyến mãi, nội dung trang chủ.

Nhóm 4 — Báo cáo. Doanh thu theo tháng, tỉ lệ lấp đầy, tỉ lệ huỷ.

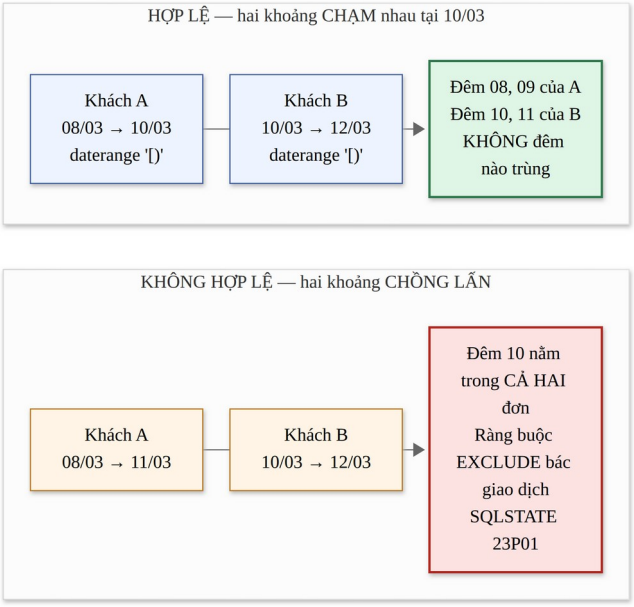
1.1.2. Bài toán cốt lõi: không được bán trùng một phòng

Đây là ràng buộc nghiệp vụ nghiêm ngặt nhất và là trục kỹ thuật của toàn đồ án. Phát biểu hình thức:

> Với mọi phòng vật lý *p*, không tồn tại hai đơn đặt phòng còn hiệu lực cùng giữ *p* mà khoảng ngày ở của chúng giao nhau.

Điểm tinh tế nằm ở chữ "giao nhau". Nó phải hiểu theo **khoảng nửa mở**. Khách A trả phòng ngày 10/03 và khách B nhận phòng ngày 10/03 là hợp lệ — buổi sáng A trả, buổi chiều B nhận. Nếu cài đặt bằng phép so sánh ngày thông thường, chỉ cần lệch một dấu bằng là hoặc mất một đêm doanh thu mỗi phòng, hoặc bán trùng.

Hình 1.1 minh hoạ sự khác biệt giữa hai khoảng chạm nhau và hai khoảng chồng lấn thật sự.

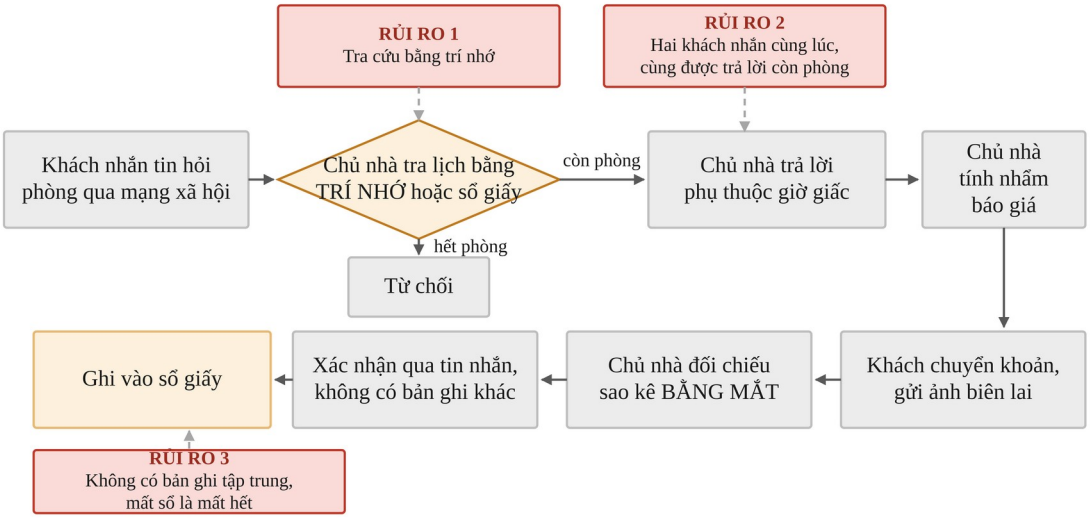


Hình 1.1 — *Sự khác biệt giữa hai khoảng chạm nhau và hai khoảng chồng lấn thật sự (Nguồn: tác giả)*

1.2. Khảo sát hiện trạng

1.2.1. Quy trình thủ công hiện tại

Hình 1.2 mô tả quy trình đặt phòng thủ công đang được áp dụng phổ biến tại các cơ sở homestay quy mô nhỏ, cùng ba điểm rủi ro chính.



Hình 1.2 — *Quy trình đặt phòng thủ công đang được áp dụng phổ biến tại các cơ sở homestay quy mô nhỏ, cùng ba điểm rủi ro chính (Nguồn: tác giả)*

Bảy bước đều có nhược điểm: khách xem ảnh trên mạng xã hội nên không biết giá và tình trạng phòng; hỏi phòng trống phụ thuộc giờ giấc chủ nhà; báo giá tính nhẩm dễ nhầm khi ở nhiều đêm hoặc nhiều phòng; đối chiếu tiền cọc làm bằng mắt trên sao kê; xác nhận không để lại bằng chứng ngoài đoạn hội thoại; dữ liệu nằm trên sổ giấy nên

không thống kê được. **Riêng bước tra lịch bằng trí nhớ là nguồn gốc của việc đặt trùng phòng.**

1.2.2. Khảo sát người dùng

Đề tài xác định **ba nhóm người dùng** với nhu cầu khác hẳn nhau.

Bảng 1.1 — *Nhu cầu của ba nhóm người dùng và hệ quả thiết kế (Nguồn: tác giả)*

Nhóm	Cần gì	Hệ quả thiết kế
Khách vãng lai — chiếm đa số	Xem phòng, biết giá, đặt được phòng; không muốn tạo tài khoản	Tài khoản là **tuỳ chọn**, không phải điều kiện. Bắt đăng ký trước khi đặt là chỗ mất khách nhiều nhất của một trang đặt phòng
Khách có tài khoản	Mọi thao tác của khách vãng lai, thêm việc xem lại toàn bộ đơn ở một nơi	Một màn hình danh sách đơn của riêng tài khoản
Quản trị viên — chủ homestay và nhân viên	Nhìn thấy toàn bộ đơn, xử lý tiền, cập nhật thông tin phòng; dùng hằng ngày	Khu quản trị ưu tiên **mật độ thông tin** cao hơn tính thẩm mỹ

1.3. Khảo sát các hệ thống tương tự

Đề tài khảo sát hai nền tảng đặt phòng lớn nhất tại thị trường Việt Nam là Booking.com và Agoda, cùng mô hình Airbnb cho phân khúc lưu trú cá nhân.

Bốn thành phần được phân tích trên trang kết quả tìm kiếm của các nền tảng này là thanh tìm kiếm cố định, thẻ phòng, tổng tiền cả kỳ và nhãn khan hiếm.

1.3.1. So sánh với hệ thống của đề tài

Khác biệt căn bản: ba nền tảng trên là **sàn trung gian nhiều cơ sở**, thu hoa hồng 15–20% mỗi đơn, còn hệ thống của đề tài phục vụ **một cơ sở duy nhất**. Đề tài giữ những điểm mạnh đã kiểm chứng của cả ba — đặt phòng không cần tài khoản, hiện tổng tiền cả kỳ ở mọi bước, chặn sẵn ngày hết phòng trong lịch — nhưng dùng chuyển khoản mã QR thay cho thẻ, hợp thói quen trong nước và không cần tài khoản cổng thanh toán quốc tế.

1.3.2. Mẫu thiết kế áp dụng và mẫu cố ý không áp dụng

Đề tài áp dụng các mẫu thiết kế đã được kiểm chứng qua nghiên cứu hành vi người dùng trên nền tảng đặt chỗ lưu trú [1], nhưng **từ chối** nhóm mẫu gây áp lực giả được phân tích trong khảo sát đối sánh hai nền tảng [10].

Cụ thể, đề tài giữ lại nhãn "còn N phòng" nhưng buộc con số đó lấy trực tiếp từ cơ sở dữ liệu, có ràng buộc loại trừ bảo chứng là đúng. Đồng hồ đếm ngược chỉ dùng cho hạn giữ chỗ **có thật** trong cơ sở dữ liệu, không phải để thúc ép. Và hệ thống **không hiển thị số người đang xem** — vì hệ thống không đo được số đó.

Luận điểm: nhãn khan hiếm trên nền tảng lớn thường không tương ứng dữ liệu thật. Đồ án giữ hình thức của chúng vì chúng giúp khách quyết định nhanh, nhưng buộc mọi con số phải kiểm chứng được — khả thi chính nhờ ràng buộc ở tầng cơ sở dữ liệu.

1.4. Xác định yêu cầu

1.4.1. Yêu cầu chức năng

Đề tài xác định **32 yêu cầu chức năng** chia ba nhóm theo tác nhân:

Bảng 1.2 — *Ba nhóm yêu cầu chức năng theo tác nhân (Nguồn: tác giả)*

Nhóm	Mã	Số lượng	Nội dung chính
A — Khách	CN-01…CN-14	14	Xem loại phòng và tiện nghi; tìm phòng theo ngày; xem lịch từng đêm; gửi yêu cầu đặt phòng; xem chi phí dự kiến; áp mã khuyến mãi; thanh toán mã QR; tra cứu và huỷ đơn; nhận thư xác nhận; viết đánh giá; đăng ký và xem lại đơn; đọc tin tức
B — Quản trị viên	CN-15…CN-28	14	Đăng nhập và buộc đổi mật khẩu tạm; quản lý đơn và chuyển trạng thái; ghi chú nội bộ; đối soát thanh toán; quản lý loại phòng, phòng vật lý, **ngày khả dụng**, khuyến mãi, đánh giá, nội dung trang chủ; xem tổng quan; xuất báo cáo CSV
C — Hệ thống tự động	CN-29…CN-32	4	Nhận webhook và tự xác nhận đơn; quét đơn quá hạn và nhả phòng; gửi thư qua hàng đợi; xử lý tiền về muộn

Danh sách chi tiết từng yêu cầu trình bày ở Phụ lục K.

1.4.2. Yêu cầu phi chức năng

Mười hai yêu cầu phi chức năng chia theo năm loại: **toàn vẹn** ba yêu cầu, **bảo mật** ba, **khả dụng** ba, **hiệu năng** một, **triển khai** một và **bảo trì** một. Mỗi yêu cầu đi kèm một cách kiểm chứng cụ thể chứ không phải một lời hứa — ví dụ "không bao giờ bán trùng một phòng" được kiểm bằng ca kiểm thử đa luồng thật, "mặc định từ chối mọi endpoint chưa khai quyền" được kiểm bằng ma trận ba mươi tiền tố đường dẫn. Bảng đầy đủ mười hai yêu cầu kèm cách kiểm chứng trình bày ở Phụ lục O.

1.5. Phạm vi và giới hạn của hệ thống

Trong phạm vi: toàn bộ 32 yêu cầu chức năng và 12 yêu cầu phi chức năng nêu trên, đã cài đặt và kiểm thử.

Ngoài phạm vi và lý do:

Bảng 1.3 — *Những chức năng hệ thống không có và lý do (Nguồn: tác giả)*

Không có	Lý do
Cổng thanh toán thật	Đề tài không yêu cầu; hệ thống viết đúng luồng thật, chỉ thiếu tài khoản thật
Hoàn tiền tự động	Hoàn tiền là quyết định kinh doanh; hệ thống đưa vào hàng đợi cho người xử lý
Ứng dụng di động	Website đã đáp ứng; xây dựng ứng dụng di động là một đề tài riêng
Nhiều chi nhánh	Bài toán khác hẳn về mô hình dữ liệu
Giao thức HTTPS trong bản đóng gói	Cần tên miền và chứng chỉ thật; đây là giới hạn đã biết, trình bày ở mục 5.7

Bảng tổng hợp toàn bộ công nghệ sử dụng theo từng tầng trình bày ở Phụ lục L.

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

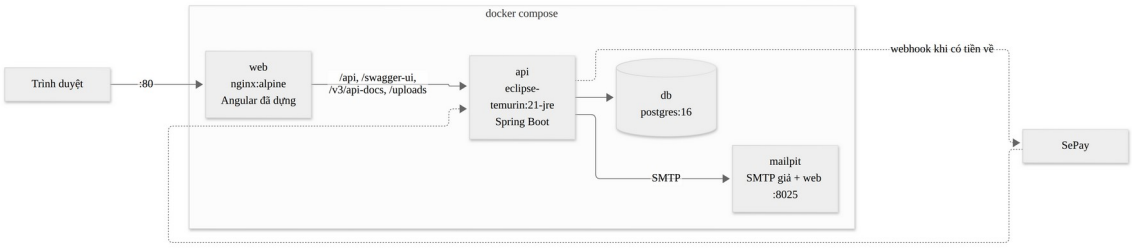
2.1. Kiến trúc ứng dụng web tách rời

2.1.1. Mô hình khách — chủ và giao diện lập trình ứng dụng REST

Hệ thống theo kiến trúc **tách rời**: tầng giao diện và tầng máy chủ là hai ứng dụng độc lập, giao tiếp qua giao diện lập trình ứng dụng trả về dữ liệu định dạng JSON. Tầng giao diện không truy cập cơ sở dữ liệu; tầng máy chủ không sinh HTML.

So với kiến trúc nguyên khối kết xuất phía máy chủ, kiến trúc tách rời phức tạp hơn lúc khởi đầu và cần thêm công sức tối ưu công cụ tìm kiếm. Đổi lại, **hai giao diện rất khác nhau** — trang bán hàng cho khách và khu quản trị mật độ cao — dùng chung **một** giao diện lập trình ứng dụng, và giao diện đó dùng lại được nguyên vẹn nếu sau này phát triển ứng dụng di động.

Hình 2.1 thể hiện kiến trúc tổng thể của hệ thống khi triển khai.



Hình 2.1 — *Kiến trúc tổng thể của hệ thống khi triển khai (Nguồn: tác giả)*

2.1.2. Nguyên tắc REST áp dụng trong đề tài

- **Tài nguyên là danh từ, thao tác là động từ HTTP.** Ví dụ `GET /api/room-types`, `POST /api/bookings`, `PATCH /api/admin/rooms/{id}`.
- **Tài nguyên con lồng vào tài nguyên cha.** Ví dụ `GET /api/admin/rooms/{id}/closures` — khoảng đóng phòng không tồn tại độc lập với phòng.
- **Mã trạng thái HTTP mang ngữ nghĩa.** Mã 201 khi tạo mới, 204 khi xoá thành công, 409 khi xung đột trạng thái, 429 khi vượt giới hạn tần suất.
- **Lỗi theo chuẩn RFC 7807 [4].** Mọi phản hồi lỗi có cấu trúc thống nhất kèm trường `code` do hệ thống tự thêm. Tầng giao diện đọc trường `code`, không đọc câu chữ mô tả, nên việc sửa câu chữ ở tầng máy chủ không làm hỏng màn hình.

2.2. Ngôn ngữ và nền tảng tầng máy chủ

2.2.1. Java 21

Java 21 là phiên bản hỗ trợ dài hạn. Đề tài sử dụng ba tính năng ngôn ngữ: kiểu `record` cho toàn bộ đối tượng truyền dữ liệu, bảo đảm tính bất biến và không thể vô tình bị sửa giữa các tầng; lớp niêm phong `sealed` cho cây ngoại lệ nghiệp vụ, giúp trình biên dịch báo ngay khi thêm một loại lỗi mới mà quên xử lý; và khối văn bản để viết các câu truy vấn SQL nhiều dòng còn đọc được.

2.2.2. Spring Boot 3.5.6

Spring Boot là nền tảng xây dựng ứng dụng Java theo mô hình cấu hình sẵn, giảm lượng cấu hình thủ công mà nhà phát triển phải viết [11]. Các mô-đun được đề tài sử dụng:

Bảng 2.1 — *Các mô-đun Spring Boot sử dụng và vai trò trong đề tài (Nguồn: tác giả)*

Mô-đun	Vai trò trong đề tài
Spring Web MVC	Tầng điều khiển, ánh xạ REST
Spring Data JPA	Truy cập dữ liệu qua Hibernate cho phần thao tác cơ bản
Spring JDBC	**Các truy vấn phòng trống viết SQL thuần** (lý do ở mục 2.4.3)
Spring Security	Xác thực bằng token, phân quyền theo vai trò
Spring Validation	Kiểm tra dữ liệu đầu vào bằng chú thích
Spring Mail	Gửi thư xác nhận
Spring Scheduling	Bộ quét đơn quá hạn, bộ gửi thư

2.2.3. Giao dịch và hành vi huỷ giao dịch

Ngoài bốn tính chất ACID quen thuộc, điểm mà đề tài buộc phải hiểu sâu là **hành vi huỷ giao dịch**. Khi PostgreSQL bác một câu lệnh vì vi phạm ràng buộc, **toàn bộ giao dịch bị đánh dấu hỏng**. Mọi câu lệnh tiếp theo trong cùng giao dịch đó trả về mã lỗi `25P02` chứ không thực thi. Spring Framework cũng đánh dấu giao dịch chỉ được huỷ khi gặp ngoại lệ thời gian chạy [12].

Hệ quả trực tiếp lên thiết kế: vòng thử gán phòng tiếp theo **bắt buộc** phải chạy trong một giao dịch mới. Chi tiết cài đặt ở mục 4.3.2.

2.2.4. Hibernate và chế độ kiểm tra lược đồ

Đề tài đặt `spring.jpa.hibernate.ddl-auto = validate` và không bao giờ dùng `update`.

Chế độ `update` để Hibernate tự sửa lược đồ: nó âm thầm thêm cột, không bao giờ xoá cột thừa và không ghi lại thay đổi nào. Sau một thời gian, lược đồ máy phát triển và máy chạy thật khác nhau mà không có cách nào biết khác ở chỗ nào.

Chế độ `validate` chỉ **kiểm tra**: mọi cột mà thực thể khai báo phải tồn tại đúng kiểu trong cơ sở dữ liệu. Ứng dụng khởi động được nghĩa là **20 thực thể** khớp hoàn toàn với lược đồ do Flyway dựng ra.

2.3. Ngôn ngữ và nền tảng tầng giao diện

Tầng giao diện dùng **Angular 21** với ba đặc điểm: component độc lập, không còn khái niệm mô-đun [2]; cơ chế phản ứng signals, đủ cho quy mô này nên đề tài không dùng thư viện quản lý trạng thái bên ngoài; và nạp lười theo tuyến, nhờ đó người truy cập trang đặt phòng không phải tải mã của mười một màn hình quản trị.

Về định kiểu, đề tài **không dùng thư viện giao diện dựng sẵn** mà tự xây thư viện component trên nền **Tailwind CSS 4**. Lý do: thư viện dựng sẵn mang theo ngôn ngữ thị giác của nó, làm trang web mang dáng dấp ứng dụng gốc chứ không giống một trang đặt phòng. Tự xây cho phép kiểm soát hoàn toàn bảng màu, khoảng cách và trạng thái.

Một khái niệm cần nắm là **tầng CSS**: Tailwind 4 đặt các lớp tiện ích trong tầng `utilities`, và một quy tắc CSS không nằm trong tầng nào luôn thắng mọi quy tắc nằm trong tầng, bất kể độ ưu tiên. Đây là nguồn gốc của một lỗi thật gặp phải trong quá trình thực hiện, trình bày ở mục 4.3.

2.4. Hệ quản trị cơ sở dữ liệu PostgreSQL 16

Toàn bộ lời giải cho bài toán cốt lõi nằm ở mục này.

2.4.1. Mức cô lập giao dịch và giới hạn của nó

PostgreSQL mặc định chạy ở mức cô lập **Read Committed** [9]: một câu `SELECT` chỉ thấy dữ liệu đã commit tại thời điểm câu lệnh đó bắt đầu chạy. Mức này **không ngăn**

được hai giao dịch song song cùng đọc rồi cùng ghi — tài liệu nêu rõ ví dụ một giao dịch cập nhật giá trị trong khi giao dịch khác đang tìm theo giá trị cũ [9].

Nâng lên **Repeatable Read** thì mọi câu lệnh trong cùng giao dịch dùng chung một ảnh chụp, ngăn được đọc ảo. Mức **Serializable** ngăn thêm bất thường tuần tự hoá, nhưng đổi lại giao dịch có thể bị bác với mã `40001` và ứng dụng phải thử lại từ đầu [9].

Điểm mấu chốt cho đề tài: **ngay cả ở mức Serializable, tài liệu PostgreSQL vẫn khuyến cáo không dựa vào kiểm tra ở tầng ứng dụng trước khi ghi** — vẫn có thể gặp vi phạm ràng buộc duy nhất do xung đột giữa các giao dịch chồng lấn, kể cả sau khi đã kiểm tra tường minh rằng khoá chưa tồn tại [9]. Khuyến nghị là dùng **ràng buộc toàn vẹn**. Đó là căn cứ lý thuyết cho toàn bộ thiết kế của đồ án.

2.4.2. Kiểu dữ liệu khoảng

PostgreSQL có kiểu `daterange` biểu diễn một khoảng ngày như một giá trị duy nhất, kèm toán tử && kiểm tra hai khoảng có giao nhau hay không [8]:

```
SELECT daterange('2026-03-08','2026-03-10','[)')
    && daterange('2026-03-10','2026-03-12','[)');   -- false, không
giao nhau
```

Ký hiệu `'[)'` là **khoảng nửa mở**: bao gồm cận dưới, không bao gồm cận trên — đúng ngữ nghĩa nghiệp vụ lưu trú, nên đề tài chọn quy ước này.

2.4.3. Ràng buộc loại trừ

```
ALTER TABLE booking_rooms
    ADD CONSTRAINT booking_rooms_no_overlap
    EXCLUDE USING gist (room_id WITH =, stay WITH &&)
    WHERE (status = 'ACTIVE');
```

Ràng buộc trên đọc là: không được phép tồn tại hai dòng có cùng `room_id` **và** có `stay` giao nhau, trong số các dòng đang ở trạng thái `ACTIVE`.

Ba lý do ràng buộc này là lời giải đúng:

Thứ nhất, nó đóng khe hở tranh chấp. Cách làm quen thuộc là kiểm tra rồi ghi:

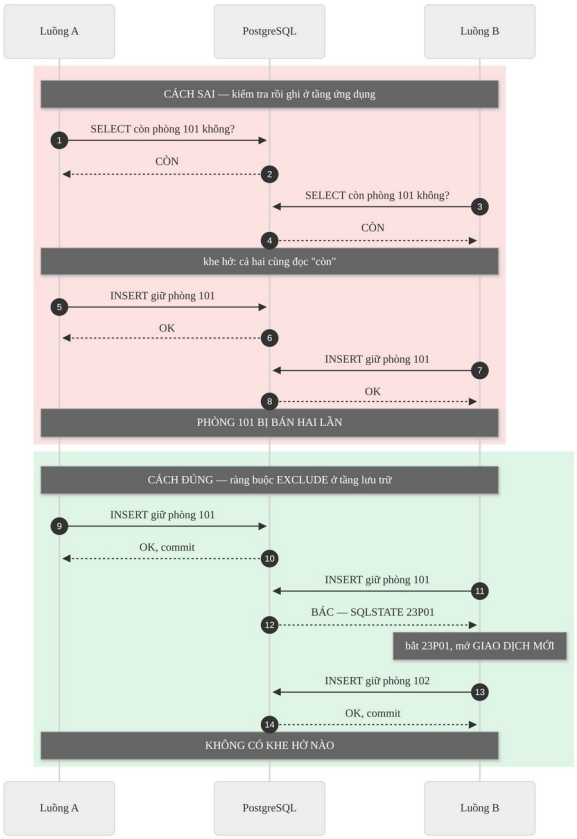
```
SELECT ... WHERE NOT EXISTS (đơn nào giao ngày không?)   -- kiểm tra
INSERT INTO booking_rooms ...                            -- rồi ghi
```

Giữa hai câu lệnh có một khoảng thời gian: hai giao dịch song song cùng đọc kết quả còn phòng rồi cùng ghi, và phòng bị bán hai lần. Mẫu này sai về nguyên lý như đã phân tích ở mục 2.4.1, không phải sai vì viết ẩu. Phương án khoá bi quan bịt được khe

hở nhưng biến mọi lượt đặt cùng một loại phòng thành hàng đợi một luồng, và không còn đúng khi ứng dụng chạy nhiều bản song song.

Với ràng buộc loại trừ, PostgreSQL bác một trong hai giao dịch ngay trong động cơ lưu trữ, bằng mã lỗi `SQLSTATE 23P01`. Không còn khe hở nào để chen vào.

Hình 2.2 so sánh trực quan hai cách xử lý tranh chấp.



Hình 2.2 — *Trực quan hai cách xử lý tranh chấp (Nguồn: tác giả)*

Thứ hai, nó đúng ngữ nghĩa nửa mở mà không cần một dòng mã ứng dụng nào.

Thứ ba, nó cho phép huỷ đơn nhả phòng ngay mà vẫn giữ lịch sử. Mệnh đề `WHERE (status = 'ACTIVE')` làm ràng buộc chỉ xét những dòng đang giữ chỗ. Huỷ đơn chỉ đổi trạng thái sang `RELEASED`: phòng mở lại lập tức, dòng dữ liệu vẫn còn để tra cứu.

2.4.4. Chỉ mục GiST và phần mở rộng btree_gist

Ràng buộc loại trừ cần một chỉ mục hỗ trợ toán tử &&. Chỉ mục B-tree thông thường được thiết kế cho dữ liệu vô hướng và **chỉ hỗ trợ so sánh tuyến tính** — bằng, lớn hơn, nhỏ hơn. Nó không trả lời được câu hỏi hai khoảng có giao nhau hay không.

GiST là viết tắt của Generalized Search Tree — cây tìm kiếm tổng quát. Tài liệu PostgreSQL định nghĩa nó là một phương thức truy cập có cấu trúc cây cân bằng, đóng vai trò khuôn mẫu cơ sở để cài đặt các lược đồ đánh chỉ mục tuỳ ý [7].

Điểm khác biệt cốt lõi so với B-tree: B-tree có cấu trúc cố định dựa trên thứ tự tuyến tính, còn GiST cho phép định nghĩa **lớp toán tử tuỳ chỉnh** cho từng kiểu dữ liệu. Nhờ đó GiST xử lý được các phép toán như giao nhau, bao hàm, khoảng cách — những phép toán không quy về được thứ tự tuyến tính [7].

Ràng buộc của đề tài trộn hai toán tử: `room_id WITH =` thuộc về B-tree và `stay WITH &&` thuộc về GiST. Phần mở rộng **`btree_gist`** cho phép đưa cả hai vào một chỉ mục GiST duy nhất. Đây là lý do migration đầu tiên của hệ thống bắt đầu bằng câu lệnh tạo phần mở rộng này.

2.4.5. Cột sinh tự động

```
stay daterange GENERATED ALWAYS AS (daterange(check_in, check_out,
'[)')) STORED
```

Cột `stay` không bao giờ được ghi trực tiếp; PostgreSQL tự tính từ hai cột ngày. Nếu để tầng ứng dụng tự tính rồi ghi vào, sẽ có lúc nó tính sai — và lúc đó ràng buộc đang canh một giá trị sai, tức là không canh gì cả.

2.4.6. Trigger ràng buộc hoãn

Hệ thống có một bất biến thứ hai: **số phòng đã gán phải khớp số phòng đơn yêu cầu**. Bất biến này không kiểm được bằng ràng buộc `CHECK` vì nó liên quan tới hai bảng.

Nếu kiểm tra ngay sau mỗi câu lệnh chèn, một đơn đặt hai phòng sẽ bị bác ngay sau dòng đầu tiên, vì lúc đó mới có một phòng trong khi đơn yêu cầu hai. Giải pháp là `CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED`: PostgreSQL hoãn việc kiểm tra tới **thời điểm commit**, khi mọi dòng đã ghi xong.

Mã nguồn đầy đủ của hai trigger này trình bày ở Phụ lục D.

2.5. Xác thực và phân quyền bằng JSON Web Token

JSON Web Token là chuỗi ký tự gồm phần đầu, phần tải và chữ ký, đã ký bằng khoá bí mật của máy chủ. Máy chủ không cần lưu phiên làm việc, chỉ cần xác minh chữ ký.

Đề tài dùng **hai loại token với hai vòng đời**: token truy cập sống ngắn, lưu trong bộ nhớ của trang và gửi kèm mỗi yêu cầu; token làm mới sống dài, lưu trong cookie mà mã kịch bản không đọc được. Tách đôi như vậy vì token truy cập có thể bị mã độc chèn vào trang lấy được, nhưng nó hết hạn sau vài phút; còn token làm mới sống lâu thì nằm ngoài tầm với của mã kịch bản.

Nhược điểm cố hữu của JWT là không thu hồi được trước hạn. Đề tài khắc phục bằng một **số phiên bản** lưu trong bảng người dùng và nhúng vào token: khi đổi mật khẩu hoặc đăng xuất, số đó tăng lên và mọi token cũ mất hiệu lực ngay.

2.6. Quản lý phiên bản lược đồ bằng Flyway

Flyway thực thi các tệp SQL đánh số theo thứ tự và ghi lại tệp nào đã chạy kèm giá trị băm của nó. Nguyên tắc **bất biến**: migration đã phát hành không bao giờ được sửa, vì sửa làm giá trị băm thay đổi và Flyway chặn khởi động. Muốn đổi lược đồ thì thêm một migration mới. Hệ thống hiện có chín migration.

Dữ liệu mẫu **cố ý không đi qua Flyway**. Nếu dữ liệu minh hoạ là một migration, việc chuyển sang môi trường thật trên cùng khối dữ liệu sẽ mang theo cả 329 đơn giả. Dữ liệu mẫu được nạp bởi một thành phần riêng chỉ kích hoạt ở cấu hình trình diễn.

2.7. Đóng gói bằng Docker và Docker Compose

Ảnh Docker là bản đóng gói gồm ứng dụng và mọi thành phần nó cần để chạy. Đề tài dùng **tệp Dockerfile nhiều tầng**: tầng đầu chứa công cụ biên dịch, tầng sau chỉ chứa kết quả biên dịch, nên ảnh cuối không mang theo trình biên dịch và có ít bề mặt tấn công hơn. Docker Compose mô tả nhiều dịch vụ trong một tệp cấu hình.

Hệ thống gồm bốn dịch vụ: máy chủ web, ứng dụng, cơ sở dữ liệu và máy chủ thư giả lập. **Chỉ máy chủ web và giao diện xem thư mở cổng ra máy chủ**; ứng dụng và cơ sở dữ liệu không mở cổng nào, và đây là điều kiện để tin được thông tin địa chỉ mà máy chủ web chuyển tiếp, giải thích ở mục 3.9.3.

2.8. Thanh toán qua mã QR và webhook

VietQR là chuẩn mã QR thanh toán của hệ thống ngân hàng Việt Nam. Quét mã điền sẵn số tài khoản, số tiền và nội dung chuyển khoản.

SePay là dịch vụ trung gian đọc biến động số dư của tài khoản ngân hàng và gọi webhook tới hệ thống khi có tiền về [14].

Webhook là cơ chế ngược với việc hỏi liên tục: thay vì hệ thống hỏi có tiền chưa mỗi vài giây, nhà cung cấp chủ động gọi vào một endpoint do hệ thống cung cấp.

Ba vấn đề kỹ thuật phải giải quyết:

Bảng 2.2 — *Các vấn đề của thanh toán qua webhook và cách giải quyết (Nguồn: tác giả)*

Vấn đề	Cách giải quyết
Bất kỳ ai cũng gọi được endpoint webhook	Xác thực bằng khoá trong phần đầu yêu cầu
Nhà cung cấp có thể gửi lại cùng một sự kiện	Ràng buộc duy nhất trên cặp nhà cung cấp và mã sự kiện — gửi lại không cộng tiền hai lần
Tiền về sau khi đơn đã hết hạn	Mở lại đơn sang trạng thái chờ đối soát, thử giành lại phòng; không được thì vào hàng đợi hoàn tiền

2.9. Kiểm thử với Testcontainers

Testcontainers khởi động một container Docker thật trong lúc chạy kiểm thử. Đề tài bắt buộc dùng nó vì phần lớn nội dung đáng kiểm thử ở hệ thống này **chính là hành vi của cơ sở dữ liệu**: ràng buộc loại trừ, kiểu khoảng, trigger hoãn, mã trạng thái SQL. Cơ sở dữ liệu trong bộ nhớ không có những tính năng đó, nên kiểm thử chạy trên nền giả lập sẽ thành công mà không chứng minh được điều duy nhất đáng chứng minh.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Phân tích chức năng — biểu đồ use case

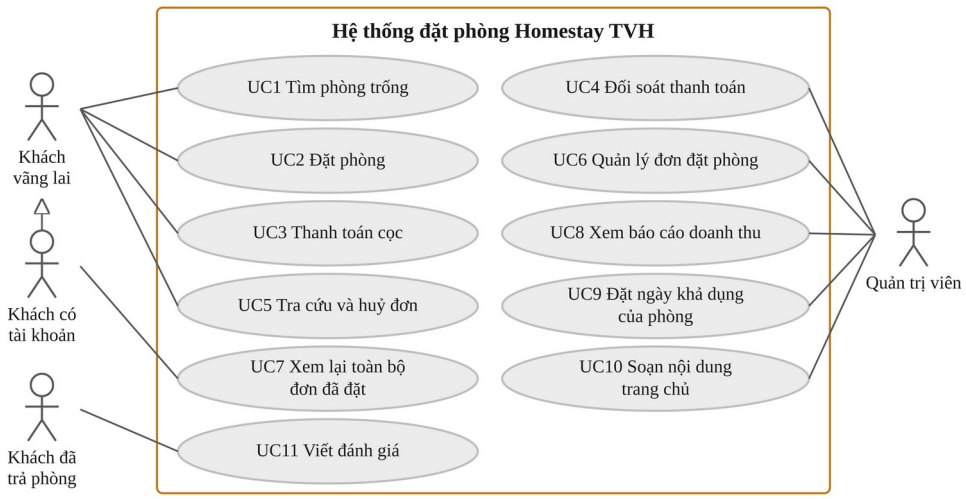
3.1.1. Xác định tác nhân

Bảng 3.1 — *Các tác nhân của hệ thống và cơ chế xác thực tương ứng (Nguồn: tác giả)*

Tác nhân	Mô tả	Cơ chế xác thực
Khách vãng lai	Không có tài khoản. Đặt, tra cứu và huỷ đơn bằng mã đơn và số điện thoại	Không đăng nhập. Thao tác trên một đơn cụ thể cần mã đơn và số điện thoại, hoặc mã truy cập trong liên kết thư xác nhận
Khách có tài khoản	Thực hiện được mọi thao tác của khách vãng lai, thêm việc xem lại mọi đơn ở một nơi	Token, vai trò `CUSTOMER`
Quản trị viên	Chủ homestay và nhân viên	Token, vai trò `ADMIN`
Tác nhân hệ thống	Nhà cung cấp thanh toán gọi webhook; bộ quét định kỳ	Khoá trong phần đầu yêu cầu, hoặc nội bộ

Hình 3.1 thể hiện biểu đồ use case tổng quát với đủ mười một use case của hệ thống.

Trong biểu đồ, *Khách có tài khoản* nối với *Khách vãng lai* bằng quan hệ kế thừa thay cho bốn đường liên kết trùng nhau: theo Bảng 3.1, tác nhân này làm được mọi thao tác của khách vãng lai và chỉ thêm UC7.



Hình 3.1 — *Biểu đồ use case tổng quát với đủ mười một use case của hệ thống (Nguồn: tác giả)*

3.1.2. Danh sách use case

Bảng 3.2 — *Danh sách use case và tác nhân chính (Nguồn: tác giả)*

Mã	Tên use case	Tác nhân chính
UC1	Tìm phòng trống	Khách vãng lai, Khách có tài khoản

Mã	Tên use case	Tác nhân chính
UC2	Đặt phòng	Khách vãng lai, Khách có tài khoản
UC3	Thanh toán cọc	Khách vãng lai, Khách có tài khoản
UC4	Đối soát thanh toán	Quản trị viên
UC5	Tra cứu và huỷ đơn	Khách vãng lai, Khách có tài khoản
UC6	Quản lý đơn đặt phòng	Quản trị viên
UC7	Xem lại toàn bộ đơn đã đặt	Khách có tài khoản
UC8	Xem báo cáo doanh thu	Quản trị viên
UC9	Đặt ngày khả dụng của phòng	Quản trị viên
UC10	Soạn nội dung trang chủ	Quản trị viên
UC11	Viết đánh giá	Khách đã trả phòng

3.1.3. Đặc tả use case

Đặc tả đầy đủ của **cả mười một use case** — tác nhân, tiền điều kiện, luồng chính, luồng thay thế, hậu điều kiện — trình bày ở Phụ lục A. Hai use case trọng tâm là UC1 *Tìm phòng trống* và UC2 *Đặt phòng*: UC1 là nơi truy vấn còn phòng chạy, UC2 là nơi ràng buộc loại trừ của cơ sở dữ liệu ra quyết định cuối cùng khi hai khách cùng đặt một phòng.

3.2. Thiết kế cơ sở dữ liệu

3.2.1. Tổng quan

Lược đồ gồm **21 bảng nghiệp vụ**, dựng bởi **chín migration** Flyway từ `V1` tới `V9`. Bảng thứ 22 trong lược đồ công khai là sổ ghi chép của chính Flyway, không thuộc mô hình nghiệp vụ.

Hình 3.2 thể hiện sơ đồ quan hệ thực thể tổng thể.

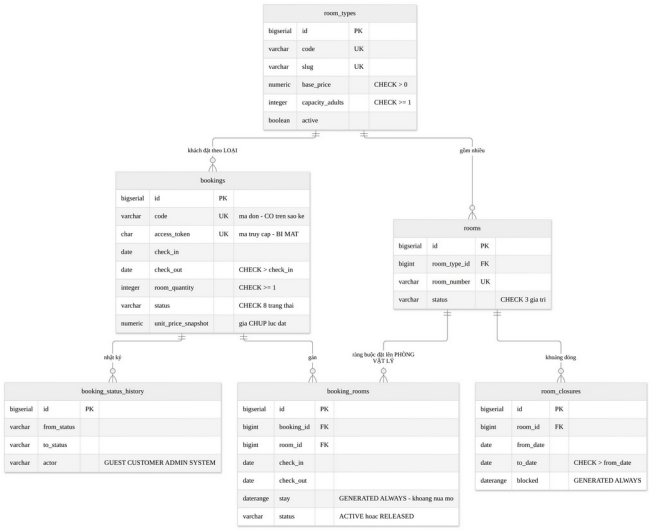


Hình 3.2 — *Sơ đồ quan hệ thực thể tổng thể (Nguồn: tác giả)*

Bảng 3.3 — *Các nhóm bảng dữ liệu theo migration (Nguồn: tác giả)*

Nhóm	Migration	Các bảng	Vai trò
Người dùng và xác thực	V1	`users`, `refresh_tokens`	Tài khoản, vai trò, phiên đăng nhập
Phòng và tiện nghi	V2	`amenities`, `room_types`, `rooms`, `room_type_images`, `room_type_amenities`	Danh mục sản phẩm
Đặt phòng	V3	`bookings`, `booking_rooms`, `booking_status_history`	**Lõi nghiệp vụ**
Thanh toán và thư	V4	`payments`, `payment_webhook_events`, `outbound_emails`	Tiền và liên lạc
Khuyến mãi và đánh giá	V5	`promotions`, `reviews`	Tiếp thị và uy tín
Nội dung	V6	`site_contents`, `banners`, `gallery_images`, `posts`	Trang chủ, tin tức
Ghi chú nội bộ	V7	`booking_notes`	Trao đổi nội bộ trên đơn
Ngày khả dụng	V8	`room_closures`	Khoảng ngày phòng không nhận khách

Hình 3.3 thể hiện chi tiết nhóm bảng đặt phòng, nơi đặt ràng buộc quan trọng nhất của hệ thống.



Hình 3.3 — *Chi tiết nhóm bảng đặt phòng, nơi đặt ràng buộc quan trọng nhất của hệ thống (Nguồn: tác giả)*

3.2.2. Mô tả bốn bảng cốt lõi

Bốn bảng dưới đây mang các ràng buộc quyết định của hệ thống; phần này giải thích **vì sao** mỗi ràng buộc tồn tại. Danh sách cột đầy đủ của cả 21 bảng, theo mẫu ba cột tên cột – kiểu – mô tả và ràng buộc, trình bày ở Phụ lục B.

Bảng `users` — tài khoản người dùng

Ràng buộc `ck_users_email_lower` buộc lưu email chữ thường **ở tầng dữ liệu** thay vì phó mặc cho tầng ứng dụng. Chỉ cần một chỗ quên chuẩn hoá là tạo được hai tài khoản cho cùng một hộp thư.

Bảng `bookings` — đơn đặt phòng

Bốn cột kết thúc bằng `_snapshot` lưu bản sao giá trị tại thời điểm đặt. Khi quản trị viên đổi giá hoặc đổi tên loại phòng sau này, đơn cũ vẫn giữ nguyên số tiền và tên mà khách đã nhìn thấy lúc đặt.

Bảng `booking_rooms` — gán phòng vật lý cho đơn

Ràng buộc quyết định của toàn hệ thống nằm trên bảng này:

```
EXCLUDE USING gist (room_id WITH =, stay WITH &&) WHERE (status =
'ACTIVE')
```

Bảng này còn mang hai trigger ràng buộc hoãn bảo đảm số dòng `ACTIVE` luôn khớp `room_quantity` của đơn, đặt ở cả hai phía: khi sửa `booking_rooms` và khi sửa `room_quantity` trên `bookings`.

Bảng `room_closures` — khoảng ngày phòng không nhận khách

Ràng buộc `room_closures_no_overlap` chống hai khoảng đóng chồng nhau trên cùng một phòng.

3.2.3. Hai lần sử dụng ràng buộc loại trừ

Cùng một cơ chế được sử dụng cho hai bài toán khác nhau, cho thấy lựa chọn PostgreSQL không nhằm phục vụ một trường hợp duy nhất.

Bảng 3.4 — *Hai lần sử dụng ràng buộc loại trừ và điều được chống (Nguồn: tác giả)*

Lần	Bảng	Ràng buộc	Chống điều gì
1	`booking_rooms`	`booking_rooms_no_overlap`	Hai đơn cùng giữ một phòng trong những ngày giao nhau
2	`room_closures`	`room_closures_no_overlap`	Hai khoảng đóng chồng nhau trên cùng một phòng

Lần thứ hai giải quyết một lỗi rất khó chẩn đoán từ phía người dùng. Nếu cho phép hai khoảng đóng chồng nhau, màn hình quản trị hiển thị hai dòng mô tả cùng một điều, và việc xoá một dòng **không mở lại được phòng**: người dùng nhấn xoá, thấy dòng biến mất, rồi vẫn không bán được phòng mà không có cách nào hiểu vì sao.

Khác biệt giữa hai ràng buộc: `booking_rooms_no_overlap` có mệnh đề điều kiện `WHERE (status = 'ACTIVE')` vì dòng gán phòng có trạng thái và cần giữ lịch sử; `room_closures_no_overlap` không có, vì khoảng đóng không có trạng thái và xoá là xoá hẳn.

3.2.4. Chuẩn hoá dữ liệu

Lược đồ đạt **dạng chuẩn 3**. Hai điểm đáng nêu:

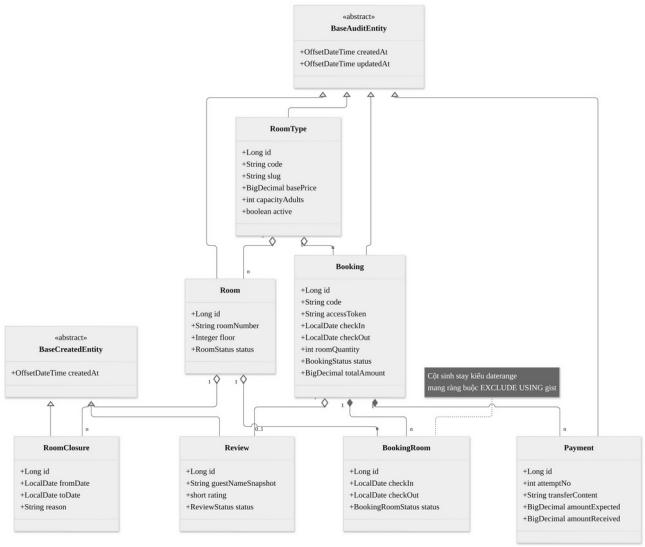
Tách `room_types` và `rooms` là quyết định mô hình hoá quan trọng nhất. Khách đặt một **loại phòng**, nhưng ràng buộc chống trùng lịch phải đặt lên **phòng vật lý**. Gộp hai khái niệm vào một bảng thì hoặc không chống trùng được, hoặc buộc khách phải chọn đúng số phòng — điều không nền tảng đặt phòng nào thực hiện.

Các chỗ cố ý phi chuẩn hoá. Cột `guest_name_snapshot` trong bảng `reviews` và bốn cột kết thúc bằng `_snapshot` trong bảng `bookings` lưu bản sao giá trị tại thời điểm phát sinh, thay vì tham chiếu. Lý do: những giá trị này phải **đóng băng** tại thời điểm đó. Khách đổi tên tài khoản hoặc quản trị viên đổi giá phòng sau này không được phép làm thay đổi nội dung đã phát sinh.

3.3. Thiết kế lớp và máy trạng thái

3.3.1. Biểu đồ lớp

Hệ thống có **20 thực thể** ánh xạ tới 20 trong số 21 bảng. Hình 3.4 thể hiện biểu đồ lớp của các thực thể cốt lõi thuộc nhóm đặt phòng và thanh toán.



Hình 3.4 — *Biểu đồ lớp của các thực thể cốt lõi thuộc nhóm đặt phòng và thanh toán (Nguồn: tác giả)*

Hai lớp cơ sở trừu tượng được dùng lại cho nhiều thực thể:

Bảng 3.5 — *Các lớp cơ sở dùng chung và nhóm thực thể sử dụng (Nguồn: tác giả)*

Lớp cơ sở	Cột cung cấp	Dùng cho nhóm bảng nào
`BaseAuditEntity`	`created_at` và `updated_at`	Bảng có vòng đời dài, có sửa đổi
`BaseCreatedEntity`	Chỉ `created_at`	Bảng chỉ ghi thêm, không bao giờ sửa

Việc tách hai lớp cơ sở là bắt buộc chứ không phải lựa chọn phong cách. Chế độ kiểm tra lược đồ báo lỗi ngay nếu một thực thể khai báo một cột mà migration không có, nên không thể dùng chung một lớp cơ sở cho cả hai nhóm bảng.

3.3.2. Máy trạng thái đơn đặt phòng

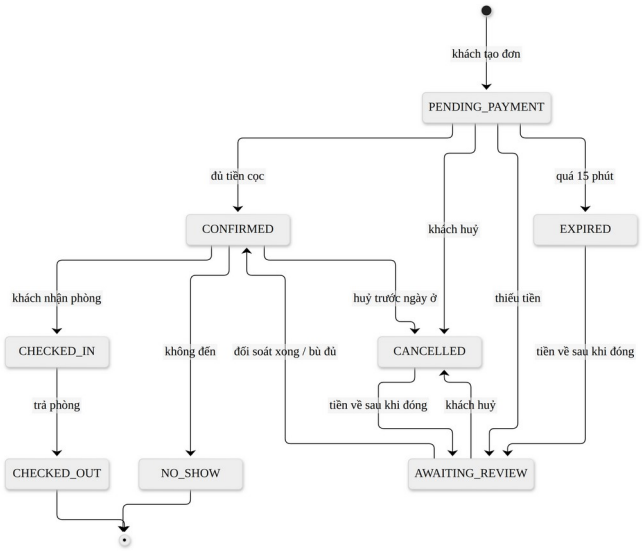
Đơn đặt phòng có **tám trạng thái**. Mọi lần chuyển trạng thái đều đi qua một thành phần duy nhất, vì mỗi lần chuyển kéo theo ba việc phụ dễ bỏ sót: ghi nhật ký, nhả phòng khi đơn không diễn ra, và hoàn lượt khuyến mãi.

Bảng 3.6 — *Tám trạng thái của đơn đặt phòng và ý nghĩa (Nguồn: tác giả)*

Trạng thái	Ý nghĩa
`PENDING_PAYMENT`	Vừa tạo, đang giữ chỗ chờ tiền cọc

Trạng thái	Ý nghĩa
AWAITING_REVIEW	Có tiền nhưng không khớp, cần người đối soát
CONFIRMED	Đã nhận đủ cọc, phòng được giữ chắc chắn
CHECKED_IN	Khách đã nhận phòng
CHECKED_OUT	Khách đã trả phòng
CANCELLED	Khách hoặc quản trị viên huỷ
EXPIRED	Quá hạn giữ chỗ mà không có tiền
NO_SHOW	Đã xác nhận nhưng khách không đến

Hình 3.5 thể hiện biểu đồ trạng thái đầy đủ, trong đó hai đường chuyển từ CANCELLED và EXPIRED về AWAITING_REVIEW là nhánh xử lý tiền về muộn.



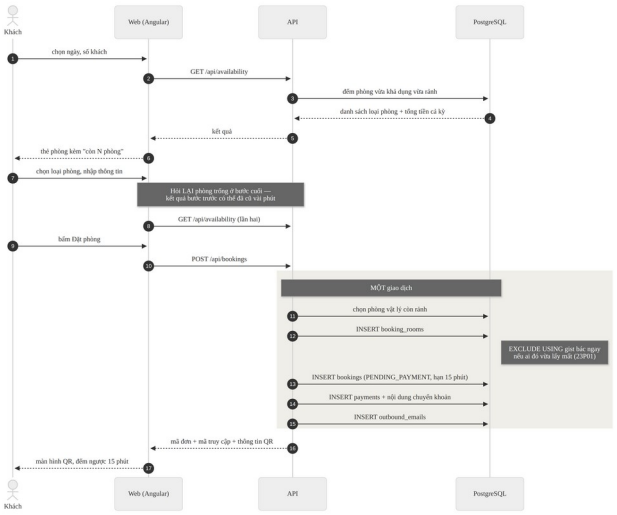
Hình 3.5 — *Biểu đồ trạng thái đầy đủ (Nguồn: tác giả)*

Một quy tắc phản trực giác cần giải thích. Trạng thái CHECKED_OUT **không** nhả phòng. Trực giác cho rằng khách trả phòng thì phòng phải được trả về kho. Nhưng nhả phòng nghĩa là chuyển dòng gán phòng sang RELEASED, trong khi trigger kiểm tra số phòng lại đòi đơn phải luôn giữ đúng số phòng đã đặt — nên nhả phòng lúc trả phòng làm cơ sở dữ liệu bác cả giao dịch. Quy tắc này phát sinh từ một lỗi thật gặp phải trong quá trình thực hiện, và có một ca kiểm thử riêng canh nó.

3.3.3. Biểu đồ tuần tự

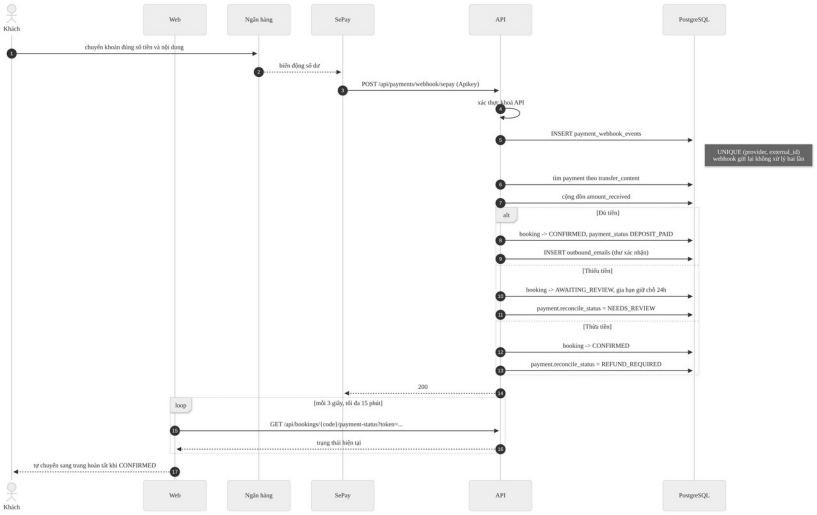
Ba luồng xử lý quan trọng nhất được mô hình hoá bằng biểu đồ tuần tự.

Hình 3.6 thể hiện luồng đặt phòng từ lúc khách chọn ngày tới lúc nhận mã QR, với bốn đối tượng tham gia: khách, trình duyệt, máy chủ ứng dụng và cơ sở dữ liệu.



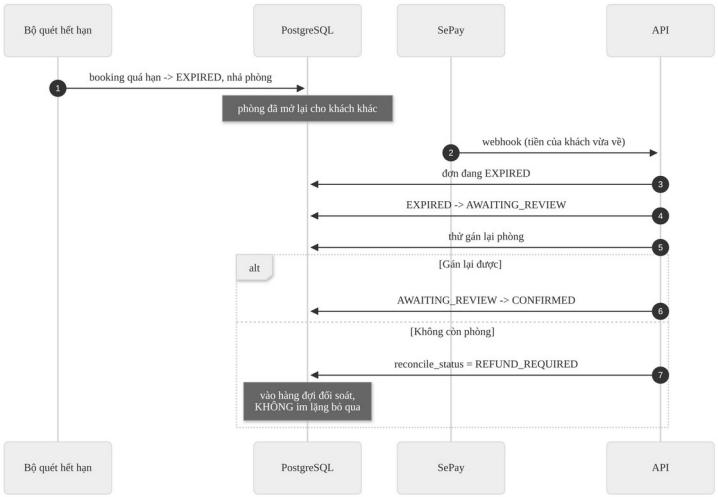
Hình 3.6 — *Luồng đặt phòng từ lúc khách chọn ngày tới lúc nhận mã QR, với bốn đối tượng tham gia: khách, trình duyệt, máy chủ ứng dụng và cơ sở dữ liệu (Nguồn: tác giả)*

Hình 3.7 thể hiện luồng thanh toán và webhook, bổ sung hai đối tượng bên ngoài là ngân hàng và nhà cung cấp dịch vụ trung gian.



Hình 3.7 — *Luồng thanh toán và webhook, bổ sung hai đối tượng bên ngoài là ngân hàng và nhà cung cấp dịch vụ trung gian (Nguồn: tác giả)*

Hình 3.8 thể hiện nhánh tiền về muộn — trường hợp tiền tới sau khi đơn đã hết hạn — cùng hai kết quả có thể xảy ra.

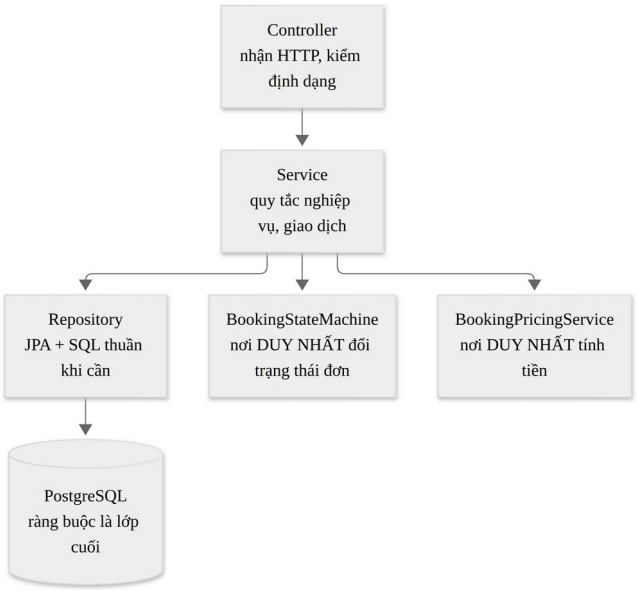


Hình 3.8 — *Nhánh tiền về muộn — trường hợp tiền tới sau khi đơn đã hết hạn — cùng hai kết quả có thể xảy ra (Nguồn: tác giả)*

3.4. Thiết kế kiến trúc phần mềm

3.4.1. Phân lớp tầng máy chủ

Hình 3.9 thể hiện sơ đồ phân lớp của tầng máy chủ.



Hình 3.9 — *Sơ đồ phân lớp của tầng máy chủ (Nguồn: tác giả)*

Bảng 3.7 — *Trách nhiệm và giới hạn của từng lớp ở tầng máy chủ (Nguồn: tác giả)*

Lớp	Trách nhiệm	Không được làm gì
Điều khiển	Nhận yêu cầu HTTP, kiểm tra định dạng đầu vào, ánh xạ sang đối tượng truyền dữ liệu	Không chứa quy tắc nghiệp vụ
Dịch vụ	Quy tắc nghiệp vụ, ranh giới giao dịch	Không biết gì về HTTP
Truy cập dữ liệu	Truy cập cơ sở dữ liệu qua JPA, và SQL thuần khi cần	Không chứa quy tắc nghiệp vụ

Lớp	Trách nhiệm	Không được làm gì
Cơ sở dữ liệu	**Lớp bảo vệ cuối cùng** bằng ràng buộc toàn vẹn	—

Hai thành phần gom về một nơi duy nhất, cùng một lý do:

- Thành phần quản lý máy trạng thái là nơi **duy nhất** đổi trạng thái đơn.
- Thành phần tính giá là nơi **duy nhất** tính tiền. Hai công thức tính tiền song song sẽ lệch nhau ngay lần đầu có khuyến mãi hoặc làm tròn, và bên lệch là bên khách nhìn thấy.

Tầng giao diện không tính tiền. Mọi con số hiển thị đều lấy từ phản hồi của máy chủ, kể cả số tiền giảm khi khách thử mã khuyến mãi. Endpoint kiểm tra mã khuyến mãi gọi lại đúng thành phần tính giá mà lúc tạo đơn sẽ sử dụng.

3.4.2. Cấu trúc gói tầng máy chủ

Mã nguồn tổ chức theo **tính năng**, không theo loại kỹ thuật: mười sáu gói dưới `com.tvh.homestay`, mỗi gói là một vùng nghiệp vụ — `auth`, `user`, `room`, `availability`, `booking`, `payment`, `promotion`, `review`, `cms`, `content`, `admin`, `report`, `storage`, `mail`, `common` và `demo`. Cây gói đầy đủ kèm chú thích từng gói trình bày ở Phụ lục P.

Lý do chọn cách tổ chức này: gom theo loại kỹ thuật khiến một thay đổi nghiệp vụ nhỏ phải sửa tệp ở ba thư mục xa nhau. Gom theo tính năng thì mọi thành phần liên quan tới đặt phòng nằm cạnh nhau.

3.4.3. Cấu trúc tầng giao diện

```
src/app
├── core/        – dịch vụ gọi máy chủ, bộ chặn yêu câù, bộ bảo vệ
tuyến
├── shared/ui/   – thư viện component tự xây
├── features/
│   ├── landing/ – trang khách: trang chủ, phòng, đặt phòng, tra cứu
│   ├── auth/    – đăng nhập, đăng ký
│   └── admin/   – 11 màn hình quản trị, nạp lười
└── styles/      – token màu, lớp nền
```

3.5. Thiết kế giao diện lập trình ứng dụng

Hệ thống phơi bày **82 thao tác trên 65 đường dẫn**, đếm từ tài liệu mô tả giao diện của hệ thống **đang chạy** chứ không lấy từ kế hoạch.

3.5.1. Nguyên tắc phân quyền: mặc định là đóng

Cấu hình bảo mật kết thúc bằng lệnh từ chối mọi yêu cầu chưa khớp quy tắc nào phía trên, nên một endpoint mới mà người viết quên khai quyền sẽ **bị từ chối** chứ không lọt ra công khai — an toàn khi thất bại thay vì mở khi thất bại.

Một lớp kiểm thử giữ một **ma trận 30 tiền tố đường dẫn** và kiểm hai điều: mọi endpoint đã đăng ký phải nằm trong ma trận, và **cấm** khai một dòng bao trùm toàn bộ khu quản trị. Việc liệt kê từng nhóm buộc người thêm endpoint mới phải cân nhắc quyền của nó, thay vì được che miễn phí.

Khu quản trị còn một lớp nữa: bộ lọc trả về lỗi `PASSWORD_CHANGE_REQUIRED` cho mọi đường dẫn trừ ba đường tối thiểu, khi tài khoản còn cờ buộc đổi mật khẩu.

3.5.2. Bảng endpoint theo nhóm

Bảng 3.8 — *Phân nhóm endpoint theo quyền truy cập (Nguồn: tác giả)*

Nhóm	Quyền	Ví dụ
Công khai — tra cứu và nội dung	Không cần đăng nhập	`GET /api/room-types`, `GET /api/availability`
Công khai — đơn đặt phòng	Mã đơn và số điện thoại, hoặc mã truy cập	`POST /api/bookings`, `POST /api/bookings/lookup`
Xác thực	Hỗn hợp	`POST /api/auth/login`, `GET /api/me`
Quản trị — đơn và thanh toán	`ADMIN`	`GET /api/admin/bookings`, `GET /api/admin/payments`
Quản trị — danh mục	`ADMIN`	`POST /api/admin/rooms/{id}/closures`
Quản trị — nội dung và đánh giá	`ADMIN`	`PUT /api/admin/content/sections/{key}`
Quản trị — báo cáo	`ADMIN`	`GET /api/admin/dashboard`
Chỉ có ở cấu hình trình diễn	—	Tài liệu giao diện tương tác

Bảng đầy đủ 82 thao tác nằm ở Phụ lục E.

3.5.3. Giới hạn tần suất

Hệ thống giới hạn tần suất theo **chín khoá** khác nhau, chia ba nhóm: theo địa chỉ mạng (`ip:auth`, `ip:availability`, `ip:promo`, `ip:booking-create`, `ip:lookup`, `ip:cancel`), theo danh tính người dùng (`email:login`, `phone:booking`) và theo mã đơn (`code:lookup`). Hạn mức đầy đủ của từng khoá trình bày ở Phụ lục Q.

Điểm thiết kế: khi hai khoá cùng áp cho một yêu cầu, **khoá chặt hơn chặn trước**. Đăng nhập sai liên tục cùng một thư điện tử bị chặn ở lần thứ sáu, còn đổi thư điện tử mỗi lần thì bị chặn ở lần thứ mười một — hai lớp bổ sung cho nhau, một lớp chặn dò mật khẩu một tài khoản, lớp kia chặn quét nhiều tài khoản.

3.5.4. Thiết kế mã lỗi

Phản hồi lỗi theo chuẩn RFC 7807 [4] kèm trường `code` tự thêm. Tầng giao diện đọc trường `code`, không đọc câu chữ mô tả, nên việc sửa câu chữ ở tầng máy chủ không làm hỏng màn hình. Hệ thống định nghĩa 15 mã lỗi nghiệp vụ, trong đó các mã quan trọng nhất là `ROOM_NOT_AVAILABLE` (409) khi phòng vừa được đặt hết, `CLOSURE_OVERLAP` (409) khi khoảng đóng phòng chồng lấn, và `INVALID_STATE_TRANSITION` (409) khi bước chuyển trạng thái không hợp lệ. Bảng đầy đủ 15 mã lỗi trình bày ở Phụ lục J.

3.6. Thiết kế giao diện người dùng

Bảng màu là **định nghĩa thương hiệu** của hệ thống: mọi mã màu tập trung ở một tệp token duy nhất, và một script trong quy trình kiểm tra chặn mã màu viết ở nơi khác. Hai lớp bổ sung nhau — xoá bảng màu mặc định của thư viện định kiểu chặn các lớp tiện ích màu sẵn có, script bắt mã màu viết thẳng vào kiểu nội tuyến.

Sáu quy ước tương tác bắt buộc, bám theo hướng dẫn khả năng tiếp cận mức A và AA [13]:

Bảng 3.9 — *Các quy ước thiết kế giao diện người dùng và lý do (Nguồn: tác giả)*

Quy ước	Lý do
Vùng chạm tối thiểu 44×44 điểm ảnh trên di động	Ngón tay không chính xác như con trỏ chuột
Chữ không nhỏ hơn 16 điểm ảnh trên di động	Dưới mức này một số trình duyệt tự phóng to khi chạm ô nhập
Lỗi không bao giờ chỉ báo bằng màu	Người khó phân biệt màu không nhận ra; luôn kèm biểu tượng và chữ
Viền chỉ báo tiêu điểm không bao giờ bị xoá	Người dùng bàn phím cần biết đang ở vị trí nào
Tôn trọng tuỳ chọn giảm chuyển động của hệ điều hành	Phục vụ người nhạy cảm với chuyển động
Chặn sẵn thay vì báo lỗi sau	Ngày hết phòng bị chặn ngay trong lịch

Về điều hướng, hệ thống chia ba khu: khu công khai không cần đăng nhập; khu tài khoản khách được bộ bảo vệ tuyến canh; và khu quản trị gồm mười một màn hình, được bộ bảo vệ tuyến canh ở tầng giao diện và kiểm tra vai trò ở tầng máy chủ. Khu

quản trị nạp lười theo tuyến để người truy cập trang đặt phòng không phải tải mã của những màn hình không sử dụng.

3.7. Môi trường và quy trình phát triển

Hệ thống phát triển bằng IntelliJ IDEA và Visual Studio Code, phụ thuộc quản lý bằng Maven và npm, mã nguồn trên Git/GitHub, cơ sở dữ liệu cục bộ chạy trong container. Yêu cầu tối thiểu để chạy: Docker kèm Compose v2 cho đường chạy khuyến nghị, hoặc JDK 21, Node.js 20.19 trở lên và **PostgreSQL 16** nếu chạy thủ công — PostgreSQL 16 là điều kiện cứng vì hệ thống cần `btree_gist`.

Đề tài thực hiện theo **mười giai đoạn** tuần tự, khoảng 97 giờ công: khởi tạo dự án (5h), hệ thống thiết kế (12h), lược đồ cơ sở dữ liệu (8h), xác thực và phân quyền (7h), lõi đặt phòng và chống trùng lịch (11h), thanh toán và webhook (10h), khu quản trị (14h), trang khách và quản lý nội dung (16h), đóng gói và tài liệu (7h), lịch khoá phòng (7h). Mỗi giai đoạn chỉ được đóng khi mọi tiêu chí nghiệm thu kiểm chứng được bằng lệnh. Hai cặp giai đoạn trong số đó thực hiện song song được vì không dùng chung tệp nào.

Quy mô mã nguồn tại thời điểm hoàn thành: 176 tệp Java với 16 044 dòng, 101 tệp TypeScript với 13 270 dòng, 9 migration cơ sở dữ liệu, 19 lớp kiểm thử với 136 ca, 59 commit trải từ 20/09/2026 tới 12/10/2026.

3.8. Cài đặt các chức năng chính

3.8.1. Chức năng tìm phòng trống

Đây là chức năng có yêu cầu kỹ thuật cao nhất ở tầng truy vấn. Hệ thống có **bốn truy vấn phòng trống** phục vụ bốn màn hình khác nhau, tất cả đặt trong một lớp truy cập dữ liệu duy nhất:

Bảng 3.10 — *Các truy vấn của chức năng tìm phòng trống (Nguồn: tác giả)*

Truy vấn	Phục vụ	Hậu quả nếu sai
Đếm phòng trống theo loại	Trang danh sách phòng, bước hai của luồng đặt phòng	Khách thấy sai số phòng còn lại
Chọn phòng vật lý để gán	**Lúc tạo đơn**	**Gán nhầm phòng — lỗi im lặng, chỉ lộ ra khi khách tới nhận phòng**
Lịch từng đêm toàn homestay	Thanh tìm phòng ở trang chủ	Lịch chặn sai ngày
Lịch từng đêm một loại phòng	Trang chi tiết phòng	Lịch chặn sai ngày

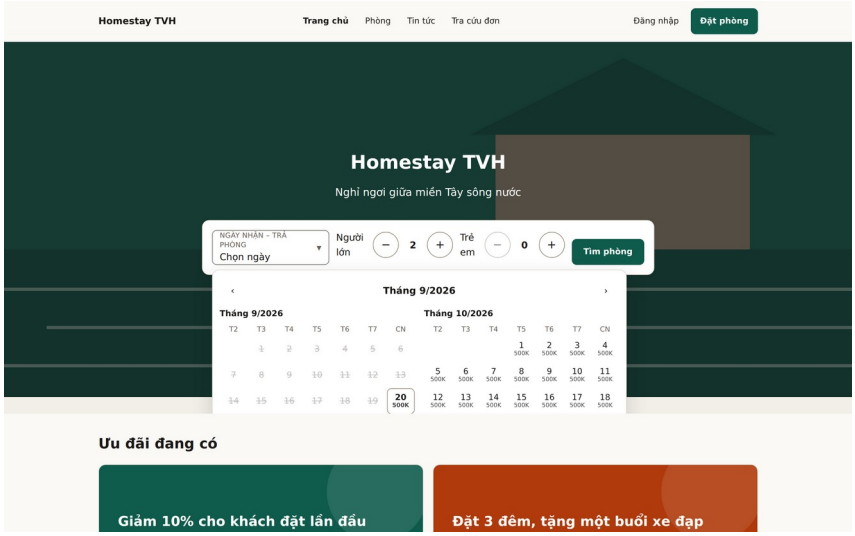
Điểm kỹ thuật thứ nhất: đếm trực tiếp thay vì lấy tổng trừ đi. Truy vấn đếm phòng trống đếm trực tiếp số phòng vừa khả dụng vừa rảnh.

Cách lấy tổng trừ đi số đã đặt sai ở chỗ nó trừ mọi đơn đang giữ chỗ của loại phòng đó, kể cả đơn nằm trên phòng đã chuyển sang bảo trì. Một phòng bảo trì đang có đơn cũ bị trừ **hai lần**: một lần vì nó không còn khả dụng, một lần nữa vì nó vẫn có đơn. Website báo thiếu phòng và mất doanh thu mà không có dấu hiệu nào.

Điểm kỹ thuật thứ hai: gộp cả khoảng ngày vào một câu lệnh. Truy vấn lịch toàn homestay dùng hàm sinh chuỗi ngày của PostgreSQL để trả về mọi đêm trong một lần gọi, thay vì lặp từng đêm. Khoảng tối đa là 120 ngày, và 120 lượt đi về cơ sở dữ liệu cho một lần mở lịch đủ để người dùng cảm nhận được độ trễ.

Điểm kỹ thuật thứ ba: vị trí của điều kiện lọc trong phép nối trái. Truy vấn lịch toàn homestay dùng phép nối trái để mọi đêm đều xuất hiện, kể cả đêm không còn phòng. Điều kiện lọc khoảng đóng phòng phải nằm trong mệnh đề nối chứ không phải mệnh đề lọc: đặt sai chỗ biến nối trái thành nối trong, những đêm hết phòng biến mất khỏi kết quả thay vì trả về số không — trong khi tầng giao diện hiểu ngày vắng mặt là ngày không bị chặn.

Hình 3.10 thể hiện trang chủ với thanh tìm phòng và lịch chọn ngày đang mở, trong đó các ngày đã kín phòng bị chặn sẵn. Trang kết quả tìm phòng hiện nhãn số phòng còn lại và tổng tiền cả kỳ của khoảng ngày đã chọn.



Hình 3.10 — *Trang chủ với thanh tìm phòng và lịch chọn ngày đang mở (Nguồn: tác giả)*

3.8.2. Chức năng đặt phòng và vòng thử gán phòng

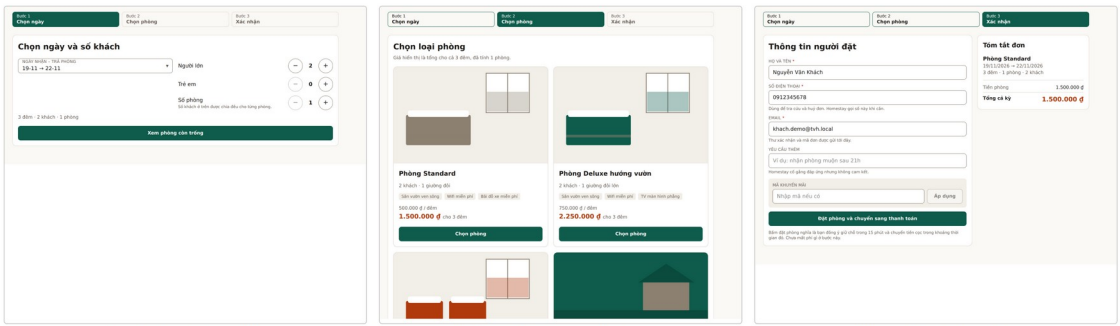
Thuật toán gán phòng là nơi lý thuyết trình bày ở Chương 2 được hiện thực hoá:

```
Với mỗi phòng ứng viên còn rảnh:
    Mở một giao dịch MỚI
    Thử ghi dòng gán phòng
    Nếu cơ sở dữ liệu trả về SQLSTATE 23P01:
        → một khách khác vừa chiếm mất phòng này
        → huỷ giao dịch, thử phòng tiếp theo
    Nếu thành công:
        → xác nhận giao dịch và kết thúc
Hết phòng để thử → trả về lỗi ROOM_NOT_AVAILABLE mã 409
```

Dòng "mở một giao dịch mới" không phải tuỳ chọn: giao dịch bị đánh dấu hỏng sau khi vi phạm ràng buộc, mọi câu lệnh sau đó trả `25P02`, nên thử phòng tiếp theo trong cùng giao dịch là **không thể thành công**.

Hệ thống cũng chỉ bắt **đúng** mã `23P01`. Bắt chung mọi lỗi toàn vẹn dữ liệu sẽ khiến một lỗi khoá ngoại — lỗi lập trình thật — bị báo cho khách thành hết phòng và không bao giờ được phát hiện.

Hình 3.11 thể hiện ba bước của luồng đặt phòng.



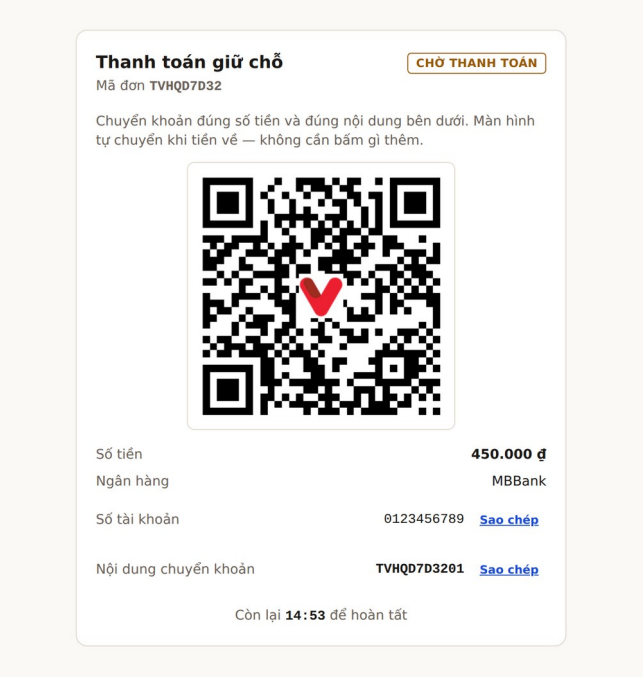
Bước 1 — chọn ngày và số khách **Bước 2** — chọn loại phòng, giá là tổng cả kỳ **Bước 3** — xác nhận và tóm tắt đơn

Hình 3.11 — *Ba bước của luồng đặt phòng (Nguồn: tác giả)*

3.8.3. Chức năng thanh toán

Hệ thống sinh mã QR theo chuẩn VietQR; nội dung chuyển khoản là **mã đơn ghép hai chữ số thứ tự lần thử** để phân biệt các lần chuyển của cùng một đơn. Đồng hồ đếm ngược 15 phút ứng với hạn giữ chỗ **có thật** trong cơ sở dữ liệu, không phải đồng hồ trang trí.

Hình 3.12 thể hiện màn hình thanh toán.



Hình 3.12 — *Màn hình thanh toán (Nguồn: tác giả)*

Xử lý webhook theo bảng quyết định sau:

Bảng 3.11 — *Các tình huống thanh toán bất thường và cách xử lý (Nguồn: tác giả)*

Tình huống	**Xử lý**
Đủ tiền cọc	Chuyển sang `CONFIRMED`, gửi thư xác nhận
Thiếu tiền	Chuyển sang `AWAITING_REVIEW`, đánh dấu cần đối soát
Thừa tiền	Đánh dấu cần hoàn lại phần thừa
Webhook gửi lại lần hai	Bỏ qua, không cộng tiền hai lần
Không khớp đơn nào	Vẫn ghi vào nhật ký webhook để người đối soát xử lý
Tiền về sau khi đơn đã đóng	Mở lại sang `AWAITING_REVIEW`, thử giành lại phòng

Nguyên tắc thiết kế xuyên suốt: không nhánh nào để tiền của khách biến mất im lặng. Mọi khoản tiền không xử lý tự động được đều vào hàng đợi đối soát thủ công, chứ không bị bỏ qua.

3.8.4. Chức năng quản lý ngày khả dụng

Trước giai đoạn mười, phòng chỉ có công tắc trạng thái vận hành, và công tắc đó **không gắn với ngày**: muốn ghi nhận một phòng sơn lại trong năm ngày thì phải tự nhớ tắt rồi tự nhớ bật, mà ngày quên bật là ngày mất doanh thu không dấu vết.

Giải pháp gồm ba phần: bảng `room_closures` với cột sinh tự động dùng cùng quy ước nửa mở, ràng buộc loại trừ chống chồng lấn, và một điều kiện lọc thêm vào **cả bốn** truy vấn phòng trống.

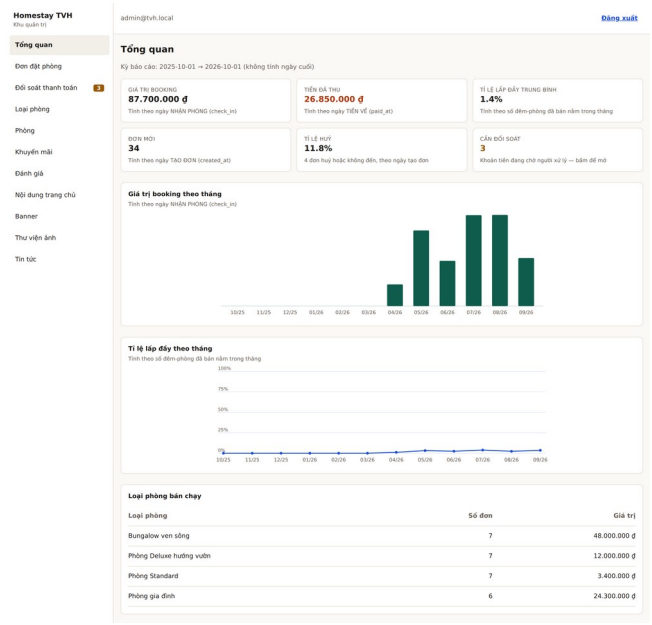
Phần thưởng kiến trúc: tầng giao diện phía khách **không phải sửa một dòng mã nào**. Cả bốn màn hình khách đều đọc phòng trống qua bốn truy vấn đó, nên sửa ở tầng dữ liệu làm cả bốn tự đúng theo — kết quả của việc gom mọi truy vấn phòng trống vào một lớp duy nhất từ giai đoạn năm.

Mã nguồn migration tạo bảng khoảng đóng phòng trình bày ở Phụ lục F.

3.8.5. Khu quản trị

Khu quản trị gồm **mười một màn hình**, nạp lười theo tuyến.

Hình 3.13 thể hiện trang tổng quan với biểu đồ doanh thu theo tháng, tỉ lệ lấp đầy và tỉ lệ huỷ. Ba màn hình còn lại của nhóm xử lý đơn là đối soát thanh toán với các khoản thiếu tiền và thừa tiền, danh sách đơn kèm bộ lọc theo trạng thái, và chi tiết một đơn — nơi dòng thời gian trạng thái, các lần thanh toán, hộp thư đi và ghi chú nội bộ hiện cùng lúc, vì đó là bốn thứ chủ homestay cần xem một lần khi có việc phải xử lý.



Hình 3.13 — *Trang tổng quan với biểu đồ doanh thu theo tháng, tỉ lệ lấp đầy và tỉ lệ huỷ (Nguồn: tác giả)*

3.8.6. Quản lý nội dung và lọc HTML

Khu quản trị cho phép soạn nội dung HTML cho trang chủ và tin tức. Đây là một bề mặt tấn công chèn mã kịch bản cổ điển [6].

Hệ thống lọc HTML **ở tầng vào**, trước khi lưu, bằng thư viện lọc của OWASP với **danh sách thẻ cho phép tường minh** — không phải danh sách thẻ cấm. Lý do: danh

sách cấm luôn thiếu; mỗi khi xuất hiện một thẻ hoặc thuộc tính nguy hiểm mới thì danh sách cấm lại lỗi thời, còn danh sách cho phép vẫn đúng.

Với nội dung do **khách** nhập, ví dụ đánh giá, hệ thống không lọc HTML mà hiển thị dưới dạng văn bản thuần, vì khách không có nhu cầu định dạng.

3.9. Cài đặt mô hình bảo mật

3.9.1. Xác thực và buộc đổi mật khẩu tạm

Tài khoản quản trị của bản trình diễn được sinh với **mật khẩu ngẫu nhiên, chỉ in một lần vào nhật ký container**, không ghi vào bất kỳ tệp nào trong mã nguồn.

Tài khoản đó mang cờ buộc đổi mật khẩu. Một bộ lọc chặn **mọi** đường dẫn trừ ba đường tối thiểu — xem thông tin bản thân, đổi mật khẩu, đăng xuất — cho tới khi mật khẩu được đổi. Lý do: mật khẩu tạm đã đi qua nhật ký nên **không còn là bí mật**.

3.9.2. Phân biệt mã đơn và mã truy cập

Một quyết định bảo mật tinh tế: hệ thống dùng **hai mã khác nhau** cho một đơn.

Mã đơn xuất hiện trên sao kê ngân hàng vì nó nằm trong nội dung chuyển khoản. Nghĩa là bất kỳ ai nhìn thấy sao kê đều biết mã đơn của người khác.

Mã truy cập là bí mật thao tác, chỉ có trong liên kết ở thư xác nhận.

Vì thế endpoint xem trạng thái thanh toán **bắt buộc** mã truy cập: nếu chỉ cần mã đơn là xem được, ai thấy sao kê cũng theo dõi được đơn của người khác.

3.9.3. Giới hạn tần suất sau máy chủ web

Ứng dụng nằm sau máy chủ web, nên mọi yêu cầu đến đều mang địa chỉ mạng của container máy chủ web. Giới hạn theo địa chỉ đó là giới hạn **toàn bộ người dùng chung một hạn mức**.

Giải pháp là để máy chủ web **ghi đè** trường tiêu đề chứa địa chỉ thật của người gọi. Nhưng trường này do phía khách gửi lên nên **giả mạo được**, trừ khi hai điều kiện cùng thoả mãn:

1. Máy chủ web **ghi đè** trường đó chứ không nối thêm vào giá trị có sẵn [3].
2. Cổng của ứng dụng **không mở ra ngoài**, nên không ai gọi thẳng vào được.

Hai điều kiện này phải đi cùng nhau; thiếu một là lớp giới hạn tần suất trở thành hình thức.

3.9.4. Quản lý bí mật

Hai biến bí mật của hệ thống **không có giá trị mặc định ở bất kỳ đâu**: không trong tệp cấu hình, không trong tệp biến môi trường mẫu, không trong tệp đóng gói. Ứng dụng **dừng khởi động** khi thiếu, ở mọi cấu hình.

Lý do: kho mã nguồn này công khai. Một giá trị mặc định được cho là an toàn cho bản trình diễn nằm trong kho đồng nghĩa với việc bất kỳ ai sao chép về cũng **tự ký được token vai trò quản trị** cho mọi bản triển khai dùng kho này.

Một script riêng là đường **duy nhất** tạo ra các giá trị thật, sinh ngẫu nhiên vào tệp biến môi trường với quyền truy cập hạn chế.

3.9.5. Các lớp bảo vệ khác

Bảng 3.12 — *Các lớp bảo vệ bổ sung của hệ thống (Nguồn: tác giả)*

Lớp	Biện pháp
Tiêu đề bảo mật	Chính sách bảo mật nội dung, chống nhúng khung, chống đoán kiểu tệp
Ảnh tải lên	Kiểm định dạng, giới hạn dung lượng, giải mã lại và đổi tên thành định danh ngẫu nhiên
Xuất tệp CSV	Thoát ký tự đầu dòng để chống chèn công thức bảng tính [5]
Mật khẩu	Băm bằng thuật toán bcrypt

3.10. Đóng gói và triển khai

3.10.1. Kiến trúc đóng gói

Bốn dịch vụ:

Bảng 3.13 — *Các dịch vụ trong tệp đóng gói Docker Compose (Nguồn: tác giả)*

Dịch vụ	Ảnh nền	Cổng mở ra máy chủ
Máy chủ web	`nginx:alpine`	**80**
Ứng dụng	`eclipse-temurin:21-jre-alpine`	Không
Cơ sở dữ liệu	`postgres:16`	Không
Máy chủ thư giả lập	`mailpit`	**8025**

Việc ứng dụng và cơ sở dữ liệu không mở cổng là một quyết định bảo mật, không phải tối giản cấu hình — đó là điều kiện để tin được thông tin địa chỉ mà máy chủ web chuyển tiếp, như đã trình bày ở mục 3.9.3.

Ảnh ứng dụng chạy bằng **người dùng không có quyền quản trị**, và được dựng bằng tệp Dockerfile nhiều tầng nên ảnh cuối không chứa công cụ biên dịch.

Nội dung đầy đủ của tệp đóng gói trình bày ở Phụ lục G.

3.10.2. Cấu hình máy chủ web

Máy chủ web phải chuyển tiếp **bốn nhóm đường dẫn** chứ không chỉ nhóm giao diện lập trình ứng dụng:

Bảng 3.14 — *Cấu hình máy chủ web và hậu quả nếu thiếu (Nguồn: tác giả)*

Đường dẫn	Hậu quả nếu quên chuyển tiếp
`/api/**`	Toàn bộ hệ thống không hoạt động
`/swagger-ui/**`	Trả về trang ứng dụng kèm mã 200 — **trông giống như thành công**
`/v3/api-docs/**`	Tương tự
`/uploads/**`	Mọi ảnh tải lên trả về lỗi 404

Cả bốn khối phải dùng cú pháp tiền tố ưu tiên. Nguyên nhân: khối cuối tệp xử lý tệp tĩnh là một khối biểu thức chính quy, và máy chủ web cho khối biểu thức chính quy thắng mọi khối tiền tố thường [3]. Không có tiền tố ưu tiên thì ảnh tải lên rơi vào khối tĩnh và trả về lỗi.

3.10.3. Hai cấu hình chạy

Hệ thống chỉ có **hai** cấu hình, cố ý không nhiều hơn:

Bảng 3.15 — *Hai cấu hình chạy của hệ thống (Nguồn: tác giả)*

Cấu hình	Sử dụng khi	Bao gồm
Trình diễn	Phát triển và bản đem bảo vệ	Dữ liệu mẫu, tài liệu tương tác, trang thư viện component
Thật	Triển khai thật	Không dữ liệu mẫu, không tài liệu tương tác, không trang thư viện

Việc đổi cấu hình trên cùng một khối dữ liệu là **an toàn** — đây chính là lý do dữ liệu mẫu không nằm trong lịch sử migration.

3.10.4. Dữ liệu mẫu

Cấu hình trình diễn nạp sẵn: 4 loại phòng, 15 phòng vật lý, 12 tiện nghi, **40 đơn đặt phòng trải đủ tám trạng thái**, 3 khoản cần đối soát, 3 mã khuyến mãi với ba tình trạng khác nhau, 8 đánh giá, 1 khoảng đóng phòng, và nội dung trang chủ đầy đủ.

Mọi ngày trong dữ liệu mẫu là tương đối so với ngày hiện tại, nên bộ dữ liệu không cũ đi theo thời gian: ngày bảo vệ vẫn có đơn trong tương lai để trình diễn.

3.10.5. Xử lý múi giờ

Toàn hệ thống chạy theo múi giờ Việt Nam, đặt ở **bốn tầng độc lập**: biến môi trường của container, tham số máy ảo Java, cấu hình chuyển đổi dữ liệu sang JSON, và cấu hình kết nối cơ sở dữ liệu.

Chỉ đặt cấu hình chuyển đổi JSON là **không đủ** — thuộc tính đó chỉ chi phối cách dữ liệu được biểu diễn khi trả về, không đổi múi giờ mặc định của máy ảo. Hệ quả nếu làm sai: đơn tạo trong khung nửa đêm tới rạng sáng giờ Việt Nam bị gom nhóm vào **tháng trước** ở trang tổng quan, trong khi đồng hồ đếm ngược trên màn hình thanh toán vẫn đúng — sai một phần nên rất khó nghi ngờ.

Endpoint kiểm tra tình trạng hệ thống phơi bày cả múi giờ ứng dụng lẫn múi giờ mặc định của máy ảo để kiểm tra được điều này trực tiếp.

3.10.6. Quy trình triển khai ba lệnh

```
git clone <địa-chỉ-kho> homestay-tvh && cd homestay-tvh
./scripts/init-env.sh          # sinh bí mật ngẫu nhiên
docker compose up -d --build   # dựng và chạy 4 dịch vụ
```

Sau lệnh thứ ba, cả bốn dịch vụ `db`, `api`, `web` và `mailpit` đều báo trạng thái `healthy`; kết quả kiểm chứng bản đóng gói ghi ở mục 4.6.

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

Chương này trình bày các kết quả đạt được sau quá trình thực hiện đồ án: kết quả kiểm thử, đánh giá hiệu năng và khả năng tiếp cận, đối chiếu với mục đích ban đầu, và các giới hạn đã biết của hệ thống.

4.1. Chiến lược kiểm thử

Hệ thống có **136 ca kiểm thử tự động trong 19 lớp**.

4.1.1. Lý do không sử dụng cơ sở dữ liệu giả lập

Mọi ca kiểm thử tích hợp dựng một **PostgreSQL 16 thật** bằng Testcontainers. Đề tài không giả lập cơ sở dữ liệu, vì phần lớn nội dung đáng kiểm thử ở hệ thống này **chính là hành vi của cơ sở dữ liệu**: ràng buộc loại trừ, kiểu khoảng, trigger hoãn, mã trạng thái SQL. Giả lập chúng là kiểm thử một thứ không tồn tại.

Container được dùng chung cho mọi lớp kiểm thử kế thừa cùng lớp nền, khởi động một lần cho cả lần chạy. Dựng lại cơ sở dữ liệu cho từng lớp kiểm thử tốn nhiều thời gian hơn phần việc thật.

Hai biến bí mật bắt buộc được **sinh ngẫu nhiên mỗi lần chạy** chứ không ghi sẵn vào tệp cấu hình kiểm thử. Cách này vừa tránh đưa giá trị vào kho mã nguồn công khai, vừa chứng minh luôn rằng ứng dụng thật sự đòi hai biến đó.

4.1.2. Các cấp độ kiểm thử

Bảng 4.1 — *Ba cấp độ kiểm thử và phạm vi kiểm của từng cấp (Nguồn: tác giả)*

Cấp độ	Ví dụ lớp	Kiểm điều gì
Kiểm thử đơn vị	Lọc HTML, xuất tệp CSV	Logic thuần, không cần cơ sở dữ liệu
Kiểm thử tích hợp	Phần lớn các lớp	Nhiều tầng cùng cơ sở dữ liệu thật
Kiểm thử tranh chấp	Đặt phòng đa luồng, đua thanh toán	Nhiều luồng chạy song song thật
Kiểm thử lược đồ	Migration, ràng buộc	Chính cấu trúc cơ sở dữ liệu

Dự án không cấu hình pha kiểm thử tích hợp riêng; công cụ chạy kiểm thử được nới để nhận cả lớp có hậu tố chỉ kiểm thử tích hợp, nên lệnh kiểm chứng gồm biên dịch, chạy toàn bộ 136 ca và đóng gói.

4.2. Kết quả kiểm thử tự động

4.2.1. Tổng hợp theo nhóm

Bảng 4.2 — *Tổng hợp số ca kiểm thử tự động theo nhóm (Nguồn: tác giả)*

Nhóm	Số lớp	Số ca	Trọng tâm
Lược đồ và ràng buộc	3	19	Migration dựng đúng 21 bảng; từng ràng buộc kiểm tra thật sự chặn
Đặt phòng	4	26	Tranh chấp đa luồng, đếm phòng trống, máy trạng thái, quét hết hạn
Ngày khả dụng	1	9	Khoảng đóng phòng trừ đúng ở cả bốn truy vấn
Thanh toán	3	22	Webhook, chống cộng tiền hai lần, cửa sổ tranh chấp, nút mô phỏng chuyển khoản
Xác thực và phân quyền	2	10	Vòng đời token, ma trận 30 tiền tố
Nội dung và đánh giá	3	32	Lọc HTML, nội dung công khai, quy tắc đánh giá
Báo cáo và lưu trữ	3	23	Số liệu tổng quan, xuất CSV, tải ảnh
Tổng	**19**	**136**	

Bảng chi tiết từng lớp kiểm thử trình bày ở Phụ lục I.

4.2.2. Ba ca kiểm thử trọng tâm

Thứ nhất, kiểm thử tranh chấp đặt phòng. Nhiều luồng thật chạy song song cùng đặt phòng cuối cùng. Kết quả bắt buộc: **đúng một luồng thắng**, không có hai dòng gán phòng chồng lấn trong cơ sở dữ liệu. Đây là bằng chứng trực tiếp cho mục đích số 2 của đề tài.

Thứ hai, kiểm thử phòng đang đóng không bao giờ được gán. Ba trong bốn truy vấn phòng trống chỉ ảnh hưởng thứ khách nhìn thấy; bỏ sót truy vấn chọn phòng vật lý thì **đơn vẫn tạo được và phòng đang sửa chữa vẫn bị gán** — lỗi im lặng, chỉ lộ ra khi khách tới nhận phòng. Ca kiểm thử này canh đúng chỗ đó.

Thứ ba, kiểm thử tiền về muộn. Tiền về **sau khi** đơn đã hết hạn: đơn mở lại sang trạng thái chờ đối soát, hệ thống thử giành lại phòng; không giành được thì vào hàng

đợi hoàn tiền. Đây là bằng chứng cho tiêu chí không nhánh nào để tiền biến mất im lặng.

4.3. Các lỗi thực tế phát hiện trong quá trình thực hiện

Mười một lỗi dưới đây đều là lỗi **đã xảy ra thật** trong quá trình làm, kèm nguyên nhân gốc chứ không phải triệu chứng.

Bảng 4.3 — *Mười một lỗi phát hiện trong quá trình thực hiện và nguyên nhân gốc (Nguồn: tác giả)*

#	Hiện tượng	Nguyên nhân gốc
1	Giao dịch thử phòng tiếp theo luôn thất bại	Bắt lỗi ràng buộc rồi thử tiếp **trong cùng giao dịch**; PostgreSQL đã huỷ giao dịch nên mọi lệnh sau trả `25P02`
2	Website báo thiếu phòng so với thực tế	Truy vấn lấy tổng trừ đi số đã đặt trừ **hai lần** phòng bảo trì đang có đơn
3	Giao dịch bị bác khi khách trả phòng	Nhả phòng lúc chuyển sang `CHECKED_OUT` làm trigger kiểm số phòng bác cả giao dịch
4	**Toàn bộ trang web hiển thị không có định kiểu**	Chính sách bảo mật nội dung chặn thuộc tính sự kiện mà công cụ tối ưu định kiểu tự sinh ra
5	**Mọi nút bấm chính có độ tương phản khoảng 2:1**	Tệp định kiểu nền **không nằm trong tầng CSS** nên thắng mọi lớp tiện ích bất kể độ ưu tiên
6	Ảnh tải lên trả về lỗi 404	Khối biểu thức chính quy của máy chủ web thắng khối tiền tố thường
7	Màn hình thanh toán trắng trơn	Component gọi hàm đọc tham số đầu vào **trong hàm khởi tạo**, trước khi khung ứng dụng gán tham số
8	Bộ nạp dữ liệu mẫu cộng dồn mỗi lần khởi động	Điều kiện chống trùng khoá theo ngày tương đối nên mỗi ngày lại khớp khác
9	Khu quản trị đóng nhầm phòng không hiển thị	Khối giao diện giữ định danh phòng cũ sau khi bộ lọc của màn hình cha thay đổi
10	**Danh sách đơn nói "bấm vào mã đơn để mở chi tiết" nhưng không bấm được**	Ô mã đơn là ô bảng thường; chỉ một liên kết ví dụ dưới bảng mở được đơn **đầu tiên**, mười chín đơn còn lại không có lối vào
11	Ô chọn tệp ảnh không có tên cho trình đọc màn hình	Ô ẩn bằng lớp chỉ-đọc-màn-hình nhưng **vẫn nằm trong thứ tự Tab**, thành một điểm dừng không tên

Cách khắc phục của từng lỗi trình bày ở Phụ lục R.

Về lỗi số 4 và số 5: cả hai chỉ phát hiện được khi **chụp màn hình bằng trình duyệt thật**. Việc biên dịch thành công không nói gì về việc trang có hiển thị đúng hay không. Hình 4.1 so sánh trước và sau khi khắc phục lỗi tương phản.



Hình 4.1 — *Trước và sau khi khắc phục lỗi tương phản (Nguồn: tác giả)*

Về lỗi số 9: lỗi này do một vòng rà soát mã độc lập tìm ra **sau khi** tác giả đã tự kiểm thử và cho rằng chức năng đã hoàn chỉnh. Đây là lý do đề tài giữ bước rà soát độc lập ở cuối mỗi giai đoạn.

Về lỗi số 10 và số 11: cả hai lộ ra trong lúc **chụp ảnh màn hình cho báo cáo**, không phải trong lúc kiểm thử. Lỗi 10 tìm ra khi một kịch bản tự động thử mở chi tiết một đơn bất kỳ và không mở được; lỗi 11 tìm ra khi mở rộng phạm vi quét khả năng tiếp cận từ sáu lên mười màn hình quản trị. Bài học giống nhau: **một màn hình chưa ai thao tác tới là một màn hình chưa được kiểm.**

4.4. Kiểm thử phi chức năng

4.4.1. Khả năng tiếp cận

Quét tự động bằng axe-core trên **17 màn hình** (7 công khai, 10 quản trị) ngay trên bản đóng gói đang chạy, đi qua đúng máy chủ web và đúng chính sách bảo mật nội dung của bản triển khai: **không còn lỗi vi phạm** hướng dẫn khả năng tiếp cận nội dung web 2.0 và 2.1 mức A và AA [13], trên tổng 356 lần quy tắc đạt.

Đo vùng chạm ở 3 độ rộng màn hình: **không còn vùng chạm dưới 44×44 điểm ảnh**.

Hình 4.2 thể hiện kết quả quét.

Kết quả quét khả năng tiếp cận tự động

Công cụ axe-core 4.13.0 · bộ quy tắc WCAG 2.0 và 2.1 mức A và AA · trình duyệt Chromium, quét trên bản đóng gói docker compose · ngày quét 20/9/2026

Màn hình	Đường dẫn	Vi phạm	Quy tắc đạt	Cần xem tay
A — Màn hình công khai (7)				
Trang chủ	`/`	0	24	1
Danh sách loại phòng	`/phong`	0	17	0
Chi tiết loại phòng	`/phong/phong-standard`	0	21	1
Luồng đặt phòng	`/dat-phong`	0	19	1
Tra cứu đơn	`/tra-cuu`	0	20	0
Đăng nhập	`/dang-nhap`	0	26	0
Tin tức	`/tin-tuc`	0	16	0
B — Màn hình quản trị (10)				
Tổng quan	`/admin`	0	23	1
Đơn đặt phòng	`/admin/bookings`	0	23	1
Chi tiết đơn	`/admin/bookings/40`	0	23	1
Đối soát thanh toán	`/admin/payments`	0	21	1
Loại phòng	`/admin/room-types`	0	21	0
Phòng và ngày khả dụng	`/admin/rooms`	0	23	1
Khuyến mãi	`/admin/promotions`	0	17	0
Đánh giá	`/admin/reviews`	0	25	1
Nội dung trang chủ	`/admin/content/sections`	0	20	0
Tin tức (quản trị)	`/admin/content/posts`	0	17	0
Tổng 17 màn hình		**0**	**356**	**9**

Cột "cần xem tay" là các mục axe-core không tự kết luận được (chủ yếu là độ tương phản trên nền ảnh); các mục này đã được kiểm tra thủ công bằng công thức tương phản WCAG.

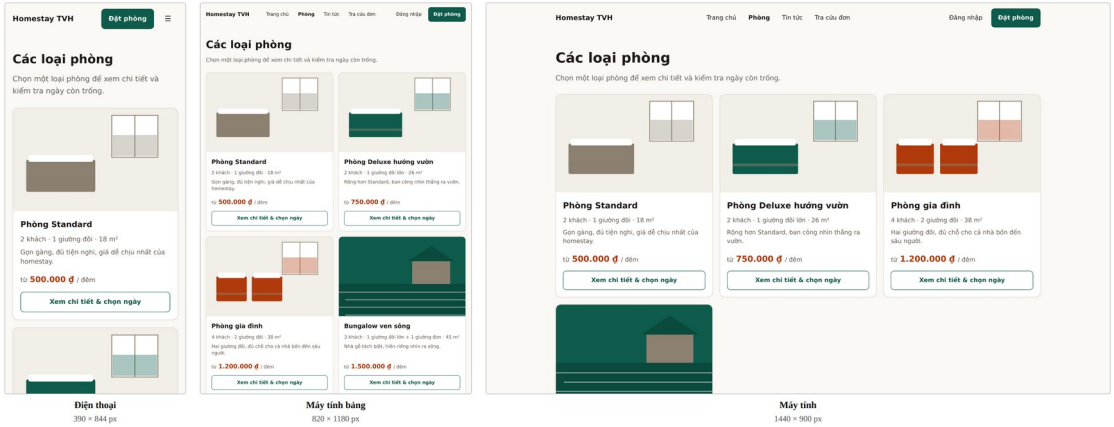
Hình 4.2 — *Kết quả quét (Nguồn: tác giả)*

4.4.2. Kiểm thử bảo mật

Chín kịch bản tấn công thử trên hệ thống đang chạy, **tất cả đều bị chặn**: giả mạo tiêu đề địa chỉ để vượt giới hạn tần suất; gọi endpoint quản trị không token hoặc bằng token vai trò khách; dùng token cũ sau khi đổi mật khẩu; gọi webhook không khoá; gửi lại cùng một webhook; chèn thẻ kịch bản vào nội dung; xem trạng thái thanh toán chỉ bằng mã đơn; truy cập đơn người khác qua tham số đường dẫn. Chi tiết ở Phụ lục M.

4.4.3. Kiểm thử đáp ứng

Hình 4.3 thể hiện giao diện trên ba kích thước màn hình: điện thoại, máy tính bảng và máy tính để bàn.



Hình 4.3 — *Giao diện trên ba kích thước màn hình: điện thoại, máy tính bảng và máy tính để bàn (Nguồn: tác giả)*

4.4.4. Kiểm thử phía tầng giao diện

Tầng giao diện **không có bộ kiểm thử tự động thường trực**. Chất lượng được canh bằng ba cổng kiểm tra:

1. Biên dịch, kiểm kiểu và kiểm mẫu hiển thị của khung ứng dụng.
2. Script kiểm tra mã màu viết trực tiếp.
3. Công cụ kiểm tra quy ước mã nguồn.

Kiểm thử giao diện trong các giai đoạn trước thực hiện bằng trình duyệt thật và được ghi lại trong báo cáo từng giai đoạn, không phải một bộ kiểm thử thường trực. **Đây là một giới hạn đã biết**, trình bày ở mục 4.7.

4.5. Kiểm thử thủ công trước khi bảo vệ

Ngoài bộ kiểm thử tự động, đề tài xác định **mười bước kiểm thử thủ công** chạy trên bản đóng gói trước khi trình diễn, phủ toàn bộ luồng nghiệp vụ chính: xem trang chủ, đặt đơn tới khi ra mã QR, tra cứu và huỷ đơn, đăng nhập quản trị và đổi mật khẩu bắt buộc, đối soát thanh toán, sửa nội dung, xem thư xác nhận, đóng phòng theo khoảng ngày và kiểm chứng số phòng giảm đúng một. Đầy đủ ở Phụ lục N.

Các lệnh dùng để kiểm chứng lại từng con số trong chương này trình bày ở Phụ lục H.

4.6. Đánh giá kết quả đạt được

4.6.1. Đối chiếu với mục đích ban đầu

Bảng 4.4 — *Đối chiếu kết quả đạt được với bốn mục đích ban đầu (Nguồn: tác giả)*

#	Mục đích	Kết quả	Bằng chứng
1	Khách tìm và đặt phòng thành công	Đạt	Kiểm thử thủ công và 6 ca tự động
2	Không bao giờ bán trùng phòng	Đạt	3 ca kiểm thử đa luồng thật
3	Thanh toán mã QR và webhook tự xác nhận	Đạt	10 ca kiểm thử webhook
4	Không để tiền biến mất im lặng	Đạt	5 ca kiểm thử đua thanh toán
5	Quản lý phòng, giá, số khách, ngày khả dụng	Đạt	9 ca kiểm thử khoảng đóng phòng
6	Giao diện đạt chuẩn thương mại	Đạt	Không còn lỗi vi phạm WCAG mức A và AA
7	Một lệnh là đủ để chạy toàn hệ thống	Đạt có điều kiện	Xem mục 4.6.3
8	Tài liệu kỹ thuật đầy	Đạt	12 tài liệu

#	Mục đích	Kết quả	Bằng chứng
	đủ		

4.6.2. Đối chiếu với yêu cầu của đề tài

Bảng 4.5 — *Mức đáp ứng các yêu cầu của đề tài (Nguồn: tác giả)*

Yêu cầu	Đáp ứng
Hiển thị phòng, tiện nghi và hình ảnh	Đạt
Tìm phòng theo ngày	Đạt
Gửi yêu cầu đặt phòng	Đạt
Quản lý trạng thái đơn	Đạt — tám trạng thái, máy trạng thái có kiểm thử
Quản trị thông tin phòng	Đạt
Quản trị viên cập nhật phòng, giá, số lượng khách và **ngày khả dụng**	Đạt đủ bốn vế
Hệ thống kiểm tra trùng ngày	Đạt — thực hiện ở tầng cơ sở dữ liệu
Tính chi phí dự kiến	Đạt — một nơi tính tiền duy nhất
Không yêu cầu thanh toán thật	Đạt — có endpoint mô phỏng
Có dữ liệu minh hoạ để trình diễn	Đạt — 329 đơn đủ tám trạng thái

4.6.3. Những điều chưa kiểm chứng được

Đề tài nêu rõ giới hạn của chính quá trình kiểm thử:

- Lần chạy đầy đủ `./mvnw verify` gần nhất cho **129 ca xanh** trên bộ kiểm thử khi đó. Bảy ca của lớp kiểm thử nút mô phỏng chuyển khoản bổ sung sau đó mới chỉ được kiểm chứng bằng cách gọi trực tiếp qua HTTP trên hệ thống đang chạy, chưa qua một lần chạy đầy đủ cả 136 ca.
- Toàn bộ bản đóng gói đã được dựng lên và kiểm chứng: bốn dịch vụ đều báo `healthy`, nginx phục vụ trang khách ở cổng 80, lời gọi `/api/room-types` đi xuyên qua nginx tới máy chủ ứng dụng trả về đủ bốn loại phòng, và thư xác nhận đi trọn đường hàng đợi thư → giao thức SMTP → hộp thư giả. Phần **chưa** kiểm chứng được chỉ còn là *tầng dựng ảnh*: môi trường phát triển có proxy chặn kho phụ thuộc Maven và npm từ bên trong container dựng ảnh, nên hai tệp `Dockerfile` được kiểm chứng ở tầng chạy với tệp jar và thư mục tĩnh dựng sẵn bên ngoài, chưa phải bằng một lệnh `docker compose up -d --build` chạy liền mạch. **Cần chạy lại đủ ba lệnh triển khai trên máy có kết nối mạng bình thường trước khi bảo vệ.**
- Tầng giao diện **không có bộ kiểm thử tự động thường trực**, như đã nêu ở mục 5.4.4. Hồi quy giao diện hiện phải phát hiện bằng mắt.

4.7. Các giới hạn đã biết của hệ thống

Bảng 4.6 — *Các giới hạn đã biết của hệ thống và hướng khắc phục (Nguồn: tác giả)*

#	Giới hạn	Ảnh hưởng	Hướng khắc phục
1	Giới hạn tần suất lưu trong bộ nhớ tiến trình	Chỉ đúng khi chạy **một** bản ứng dụng	Chuyển sang bộ nhớ đệm dùng chung khi mở rộng
2	Webhook xác thực bằng khoá tĩnh, không phải chữ ký	Khoá lộ thì giả mạo được	Dùng chữ ký băm khi nhà cung cấp hỗ trợ
3	Không có token chống giả mạo yêu cầu liên trang	Rủi ro thấp vì token nằm trong tiêu đề, không phải cookie	Bổ sung nếu chuyển sang xác thực bằng cookie
4	Hoàn tiền là thao tác thủ công	Người quản trị phải tự chuyển khoản lại	Tích hợp giao diện hoàn tiền của nhà cung cấp
5	Không có nhật ký kiểm toán cho thao tác quản trị	Không truy được ai đã sửa gì	Bổ sung bảng nhật ký kiểm toán
6	Mật khẩu chỉ yêu cầu độ dài, không yêu cầu độ phức tạp	Người dùng đặt được mật khẩu yếu	Bổ sung kiểm tra độ mạnh
7	**Không có giao thức bảo mật HTTPS trong bản đóng gói**	Mật khẩu và token đi qua mạng dạng rõ	Bổ sung chứng chỉ khi có tên miền

Giới hạn số 7 là quan trọng nhất nếu đưa hệ thống ra sử dụng thật. Bản đóng gói phục vụ giao thức không mã hoá vì nó chạy trên máy cục bộ khi trình diễn.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài đã xây dựng hoàn chỉnh một website giới thiệu và đặt phòng homestay gồm ba khối: trang bán hàng cho khách, khu quản trị cho chủ homestay, và giao diện lập trình ứng dụng với 82 thao tác trên 65 đường dẫn. Hệ thống chạy được bằng ba lệnh, kèm dữ liệu minh hoạ đủ để trình diễn mọi trạng thái nghiệp vụ.

Về mặt kỹ thuật, kết quả đáng kể nhất không phải số lượng chức năng mà là cách giải bài toán chống đặt trùng phòng: đặt ràng buộc ở tầng cơ sở dữ liệu bằng ràng buộc loại trừ trên kiểu khoảng, thay vì kiểm tra ở tầng ứng dụng. Lựa chọn này có căn cứ lý thuyết từ chính tài liệu PostgreSQL, vốn khuyến cáo không dựa vào việc kiểm tra ở tầng ứng dụng trước khi ghi ngay cả ở mức cô lập cao nhất [9]. Lời giải được chứng minh bằng kiểm thử đa luồng thật trên PostgreSQL thật, không phải bằng lập luận.

Cùng cơ chế đó được sử dụng lần thứ hai cho một bài toán khác — chống chồng lấn khoảng đóng phòng — cho thấy lựa chọn công nghệ không nhằm phục vụ một trường hợp duy nhất.

Về mặt quy trình, đề tài cho thấy giá trị của việc kiểm chứng bằng bằng chứng thay vì bằng khẳng định. Chín lỗi thật được ghi lại ở mục 4.3, trong đó có những lỗi mà việc biên dịch thành công hoàn toàn không phát hiện được.

Những nội dung đã tiếp thu được qua quá trình thực hiện:

- Mẫu kiểm tra rồi ghi ở tầng ứng dụng **không giải được** bài toán tranh chấp. Đây là hiểu biết mang tính nguyên lý, áp dụng lại được cho mọi bài toán đặt chỗ theo khoảng.
- Hành vi huỷ giao dịch của hệ quản trị cơ sở dữ liệu quan hệ ảnh hưởng trực tiếp lên thiết kế mã ứng dụng, không phải chi tiết có thể bỏ qua.
- Kiểm thử phải chạy trên hạ tầng thật khi đối tượng cần kiểm thử chính là hành vi của hạ tầng đó.
- Việc biên dịch thành công không nói gì về việc giao diện có hiển thị đúng hay không.

5.2. Hướng phát triển

5.2.1. Ngắn hạn — cần thực hiện trước khi đưa vào sử dụng thật

1. Bổ sung giao thức bảo mật HTTPS bằng chứng chỉ miễn phí.
2. Bổ sung nhật ký kiểm toán cho thao tác quản trị.
3. Kết nối tài khoản dịch vụ thanh toán thật.
4. Thay dữ liệu mẫu bằng dữ liệu thật của cơ sở lưu trú.

5.2.2. Trung hạn

1. Chuyển giới hạn tần suất sang bộ nhớ đệm dùng chung để chạy được nhiều bản ứng dụng song song.
2. Xây dựng bộ kiểm thử giao diện tự động để chống hồi quy.
3. Bổ sung bảng giá theo mùa. Hiện giá cố định theo loại phòng, chưa có giá cuối tuần hay giá ngày lễ.
4. Đồng bộ lịch hai chiều với các sàn đặt phòng qua chuẩn lịch mở, để cơ sở vừa bán trên sàn vừa bán trên website riêng mà không đặt trùng.

5.2.3. Dài hạn

1. Xây dựng ứng dụng di động sử dụng lại nguyên giao diện lập trình ứng dụng hiện có.
2. Mở rộng thành nền tảng phục vụ nhiều chi nhánh.
3. Gợi ý phòng dựa trên lịch sử đặt của khách.

DANH MỤC TÀI LIỆU THAM KHẢO

Ghi theo định dạng **IEEE** như quy định. Chỉ bao gồm các tài liệu được trích dẫn, sử dụng và đề cập tới để bàn luận trong đồ án. Đánh số theo thứ tự từ điển của tên tác giả hoặc tổ chức.

Tiếng Anh

[1] Baymard Institute, *Travel Accommodations 2024 Benchmark*, 2024. [Trực tuyến]. Có tại: https://baymard.com/blog/travel-accommodations-2024-benchmark

[2] Google LLC, *Angular Documentation*, 2026. [Trực tuyến]. Có tại: https://angular.dev

[3] Nginx Inc., *Module ngx_http_proxy_module*, 2026. [Trực tuyến]. Có tại: https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[4] M. Nottingham, E. Wilde và S. Dalal, *RFC 7807 — Problem Details for HTTP APIs*, Internet Engineering Task Force, 2016.

[5] OWASP Foundation, *CSV Injection*, 2026. [Trực tuyến]. Có tại: https://owasp.org/www-community/attacks/CSV_Injection

[6] OWASP Foundation, *OWASP Top Ten*, 2021. [Trực tuyến]. Có tại: https://owasp.org/www-project-top-ten/

[7] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation — GiST Indexes: Introduction*, 2026. [Trực tuyến]. Có tại: https://www.postgresql.org/docs/16/gist-intro.html

[8] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation — Range Types*, 2026. [Trực tuyến]. Có tại: https://www.postgresql.org/docs/16/rangetypes.html

[9] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation — Transaction Isolation*, 2026. [Trực tuyến]. Có tại: https://www.postgresql.org/docs/16/transaction-iso.html

[10] Snappymob, *UI/UX Audit: Booking.com vs Agoda*, 2023. [Trực tuyến]. Có tại: https://blog.snappymob.com/ui-ux-audit-booking-com-vs-agoda

[11] VMware Inc., *Spring Boot Reference Documentation*, 2026. [Trực tuyến]. Có tại: https://docs.spring.io/spring-boot/index.html

[12] VMware Inc., *Spring Framework Reference — Rolling Back a Declarative Transaction*, 2026. [Trực tuyến]. Có tại: https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/rolling-back.html

[13] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.1*, 2018. [Trực tuyến]. Có tại: https://www.w3.org/TR/WCAG21/

Tiếng Việt

[14] SePay, *Tài liệu tích hợp Webhooks*, 2026. [Trực tuyến]. Có tại: https://docs.sepay.vn/tich-hop-webhooks.html

PHỤ LỤC

Phụ lục A — Đặc tả mười một use case

Đặc tả đầy đủ của cả mười một use case, theo cùng một khuôn mẫu và sắp xếp theo số hiệu: tác nhân chính, mô tả, tiền điều kiện, hậu điều kiện, luồng sự kiện chính và các luồng thay thế kèm mã lỗi tương ứng.

UC1 — Tìm phòng trống

Mục	Nội dung
Mã	UC1
Tác nhân chính	Khách vãng lai, Khách có tài khoản
Mô tả	Khách tìm các loại phòng còn chỗ trong một khoảng ngày
Tiền điều kiện	Có ít nhất một loại phòng đang bật bán
Hậu điều kiện	Không có dữ liệu nào được ghi; đây là truy vấn chỉ đọc

Luồng sự kiện chính

1. Khách mở trang chủ; thanh tìm phòng nạp sẵn lịch của toàn homestay.
2. Khách chọn ngày nhận và ngày trả. Ngày quá khứ và ngày đã kín mọi loại phòng bị chặn ngay trong lịch, không cho chọn.
3. Khách chọn số người lớn, trẻ em, số phòng.
4. Hệ thống truy vấn và trả về các loại phòng còn chỗ, kèm tổng tiền cả kỳ và số phòng còn lại của từng loại.

Luồng thay thế

- *2a. Lịch không tải được:* khách vẫn chọn được ngày; mất phần chặn trước, tầng máy chủ vẫn từ chối ngày kín ở bước sau.
- *4a. Không loại phòng nào đủ chỗ:* hiện trạng thái rỗng kèm gợi ý đổi ngày hoặc giảm số khách mỗi phòng.

UC2 — Đặt phòng

Mục	Nội dung
Mã	UC2
Tác nhân chính	Khách vãng lai, Khách có tài khoản
Mô tả	Khách gửi yêu cầu đặt phòng cho một khoảng ngày và nhận mã đơn
Tiền điều kiện	Đã hoàn tất UC1 và chọn được một loại phòng còn chỗ

Mục	Nội dung
Hậu điều kiện	Đơn ở trạng thái `PENDING_PAYMENT`; các phòng vật lý đã được gán; hạn giữ chỗ 15 phút bắt đầu chạy

Luồng sự kiện chính

1. Khách nhập họ tên, số điện thoại, thư điện tử và ghi chú nếu có.
2. Khách nhập mã khuyến mãi tuỳ chọn; hệ thống kiểm tra và hiển thị ngay số tiền được giảm.
3. Khách xác nhận đơn.
4. Hệ thống mở một giao dịch, tính lại chi phí ở phía máy chủ, không tin số tiền do trình duyệt gửi lên.
5. Hệ thống chọn các phòng vật lý còn rảnh và ghi vào bảng gán phòng.
6. Cơ sở dữ liệu kiểm tra ràng buộc loại trừ. Nếu không vi phạm, giao dịch được commit.
7. Hệ thống sinh mã đơn, mã truy cập, thông tin thanh toán và trả về cho khách.
8. Hệ thống đưa thư xác nhận vào hàng đợi gửi.

Luồng thay thế và ngoại lệ

- *4a. Dữ liệu không hợp lệ* (ngày trả không sau ngày nhận, số khách vượt sức chứa mỗi phòng) → trả lỗi `INVALID_BOOKING_REQUEST` mã 400, không ghi gì.
- *2a. Mã khuyến mãi không hợp lệ* → `INVALID_PROMOTION` mã 400.
- *2b. Mã khuyến mãi hết lượt* → `PROMOTION_EXHAUSTED` mã 409.
- **6a. Ràng buộc loại trừ bị vi phạm** — một khách khác vừa đặt xong phòng đó trong tích tắc. Cơ sở dữ liệu trả `SQLSTATE 23P01`, giao dịch bị huỷ. Hệ thống mở **giao dịch mới** và thử phòng vật lý tiếp theo. Hết phòng để thử thì trả `ROOM_NOT_AVAILABLE` mã 409.
- *5a. Vượt giới hạn tần suất* → `TOO_MANY_REQUESTS` mã 429.

UC3 — Thanh toán cọc

Mục	Nội dung
Mã	UC3
Tác nhân chính	Khách vãng lai, Khách có tài khoản
Mô tả	Khách chuyển khoản tiền cọc theo mã QR và hệ thống tự xác nhận
Tiền điều kiện	Đơn đang ở `PENDING_PAYMENT` và còn trong hạn giữ chỗ

Mục	Nội dung
Hậu điều kiện	Đơn chuyển sang `CONFIRMED` nếu đủ tiền

Luồng sự kiện chính

1. Hệ thống hiển thị mã QR kèm thông tin chuyển khoản dạng chữ và đồng hồ đếm ngược tương ứng hạn giữ chỗ thật.
2. Khách quét mã và chuyển khoản.
3. Nhà cung cấp thanh toán phát hiện biến động số dư và gọi webhook.
4. Hệ thống xác thực khoá, đối chiếu nội dung chuyển khoản với mã đơn và số tiền.
5. Đủ tiền cọc, hệ thống chuyển đơn sang `CONFIRMED` và gửi thư xác nhận.
6. Trình duyệt hỏi trạng thái định kỳ và tự chuyển sang trang hoàn tất.

Luồng thay thế

- *4a. Thiếu tiền* → đơn sang `AWAITING_REVIEW`, đánh dấu cần đối soát.
- *4b. Thừa tiền* → đánh dấu cần hoàn lại phần thừa.
- *4c. Webhook gửi lại lần hai* → bỏ qua, không cộng tiền hai lần.
- *4d. Không khớp đơn nào* → vẫn ghi vào nhật ký webhook cho người đối soát.
- *4e. Sai khoá xác thực* → từ chối.
- **3a. Tiền về sau khi đơn đã hết hạn** → mở lại đơn sang `AWAITING_REVIEW`, thử giành lại phòng; không giành được thì vào hàng đợi hoàn tiền.
- *2a. Hết hạn giữ chỗ mà không có tiền* → bộ quét chuyển đơn sang `EXPIRED` và nhả phòng.

UC9 — Đặt ngày khả dụng của phòng

Mục	Nội dung
Mã	UC9
Tác nhân chính	Quản trị viên
Mô tả	Đóng một phòng vật lý trong một khoảng ngày; hệ thống tự mở lại khi hết khoảng
Tiền điều kiện	Đã đăng nhập vai trò `ADMIN`, đã đổi mật khẩu tạm
Hậu điều kiện	Phòng biến mất khỏi kết quả tìm phòng trong đúng khoảng ngày đó; **không đơn nào bị huỷ**

Luồng sự kiện chính

1. Quản trị viên mở màn hình quản lý phòng và chọn một phòng vật lý.
2. Hệ thống hiển thị các khoảng đóng còn hiệu lực của phòng đó.
3. Quản trị viên chọn ngày bắt đầu đóng và ngày mở bán lại, nhập lý do.

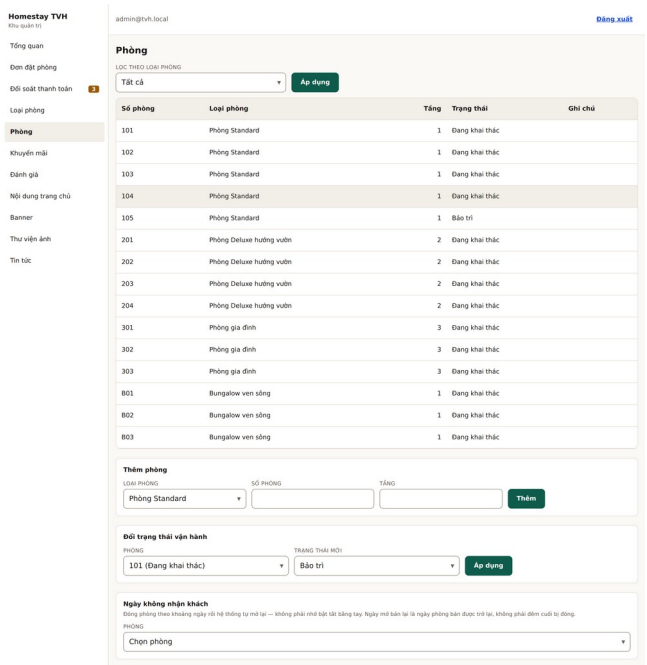
4. Hệ thống ghi khoảng đóng và cơ sở dữ liệu kiểm tra ràng buộc chống chồng lấn.
5. Hệ thống trả về danh sách các đơn đang giao với khoảng vừa đóng, chỉ để cảnh báo.

Luồng thay thế

- *3a. Ngày mở bán lại không sau ngày bắt đầu* → `INVALID_ADMIN_REQUEST` mã 400.
- *4a. Khoảng mới chồng lên một khoảng đã có* → `CLOSURE_OVERLAP` mã 409.
- *5a. Không có đơn nào trong khoảng* → không hiện cảnh báo.

Điểm thiết kế đáng lưu ý. Hệ thống **cố ý không chặn** việc đóng một phòng đang có đơn. Nếu chặn, chủ homestay không ghi nhận được sự thật rằng phòng hỏng từ ngày mai chỉ vì hệ thống còn một đơn cũ — trong khi sự thật đó vẫn xảy ra dù hệ thống có cho ghi hay không. Vai trò của hệ thống là **liệt kê** đúng các đơn bị ảnh hưởng để người quyết định nhìn thấy, không phải quyết định thay.

Hình A.1 thể hiện màn hình quản lý ngày khả dụng.



Hình A.1 — *Màn hình quản lý ngày khả dụng (Nguồn: tác giả)*

UC4 — Đối soát thanh toán

Mục	**Nội dung**
Mã	UC4
Tác nhân chính	Quản trị viên
Mô tả	Xử lý các khoản tiền không khớp tự động được

Mục	Nội dung
Tiền điều kiện	Đã đăng nhập vai trò `ADMIN`, đã đổi mật khẩu tạm
Hậu điều kiện	Khoản tiền chuyển sang trạng thái đã xử lý

Luồng sự kiện chính

1. Quản trị viên mở màn hình đối soát, thấy danh sách các khoản cần xử lý.
2. Chọn một khoản, xem chi tiết đơn và lịch sử thanh toán.
3. Quyết định: xác nhận thủ công, hoặc đánh dấu đã xử lý.
4. Hệ thống ghi lại quyết định và cập nhật trạng thái đối soát.

Luồng thay thế

- *3a. Khoản thừa tiền* → đánh dấu cần hoàn tiền; **hệ thống không tự hoàn tiền**, việc chuyển khoản lại do người thực hiện bên ngoài.
- *3b. Khoản không khớp đơn nào* → giữ trong nhật ký để tra cứu.

UC5 — Tra cứu và huỷ đơn

Mục	Nội dung
Mã	UC5
Tác nhân chính	Khách vãng lai, Khách có tài khoản
Mô tả	Khách tra cứu đơn và huỷ đơn mà không cần tài khoản
Tiền điều kiện	Khách có mã đơn và số điện thoại đã dùng khi đặt, hoặc có mã truy cập
Hậu điều kiện	Với thao tác huỷ: đơn sang `CANCELLED`, các phòng được nhả

Luồng sự kiện chính

1. Khách nhập mã đơn và số điện thoại.
2. Hệ thống xác thực và trả về thông tin đơn.
3. Khách chọn huỷ đơn.
4. Hệ thống chuyển trạng thái, nhả phòng và hoàn lượt khuyến mãi nếu có.

Luồng thay thế

- *2a. Mã đơn sai hoặc số điện thoại không khớp* → trả về **cùng một thông báo lỗi** `BOOKING_NOT_FOUND` cho cả hai trường hợp, để không ai dò được mã đơn nào có thật.
- *3a. Trạng thái không cho phép huỷ* → `INVALID_STATE_TRANSITION` mã 409.
- *1a. Vượt giới hạn tần suất tra cứu* → `TOO_MANY_REQUESTS` mã 429.

UC6 — Quản lý đơn đặt phòng

Mục	Nội dung
Mã	UC6
Tác nhân chính	Quản trị viên
Mô tả	Theo dõi và chuyển trạng thái đơn theo máy trạng thái
Tiền điều kiện	Đã đăng nhập vai trò `ADMIN`
Hậu điều kiện	Trạng thái đơn thay đổi và một dòng lịch sử được ghi

Luồng sự kiện chính

1. Quản trị viên mở danh sách đơn, lọc theo trạng thái và khoảng ngày.
2. Mở chi tiết một đơn: thấy lịch sử trạng thái, các lần thanh toán, thư đã gửi và ghi chú nội bộ.
3. Chọn trạng thái mới trong số các trạng thái hợp lệ.
4. Hệ thống kiểm tra bước chuyển, ghi nhật ký, và thực hiện các việc kèm theo như nhả phòng hoặc hoàn lượt khuyến mãi.

Luồng thay thế

- *3a. Bước chuyển không hợp lệ* → `INVALID_STATE_TRANSITION` mã 409.
- *4a. Chuyển sang `CHECKED_OUT`* → **không nhả phòng**, xem giải thích ở mục 3.3.2.

UC7 — Xem lại toàn bộ đơn đã đặt

Mục	Nội dung
Mã	UC7
Tác nhân chính	Khách có tài khoản
Mô tả	Khách đã đăng nhập xem lại toàn bộ đơn đã đặt ở một nơi
Tiền điều kiện	Đã đăng nhập với vai trò `CUSTOMER`
Hậu điều kiện	Không ghi dữ liệu; truy vấn chỉ đọc

Luồng sự kiện chính

1. Khách đăng nhập và mở trang danh sách đơn đã đặt.
2. Hệ thống trả về danh sách đơn có phân trang, mặc định 10 đơn mỗi trang.
3. Khách mở chi tiết một đơn bằng mã đơn.

Luồng thay thế

- *2a. Chưa có đơn nào* → hiện trạng thái rỗng kèm liên kết tới trang đặt phòng.

- *3a. Mã đơn không thuộc về tài khoản đang đăng nhập* → từ chối.

Điểm thiết kế đáng lưu ý. Danh sách đơn được lọc theo định danh lấy từ **token của phiên**, không theo tham số trên đường dẫn. Nếu lọc theo tham số, người dùng chỉ cần đổi số trên thanh địa chỉ là xem được đơn của người khác. Số phần tử mỗi trang cũng bị giới hạn trần ở phía máy chủ để một yêu cầu không kéo về toàn bộ bảng.

UC8 — Xem báo cáo doanh thu

Mục	Nội dung
Mã	UC8
Tác nhân chính	Quản trị viên
Mô tả	Xem tổng quan doanh thu, tỉ lệ lấp đầy và tỉ lệ huỷ theo tháng
Tiền điều kiện	Đã đăng nhập vai trò `ADMIN`
Hậu điều kiện	Không ghi dữ liệu

Luồng sự kiện chính

1. Quản trị viên mở trang tổng quan.
2. Hệ thống tính khoảng thời gian mặc định là một số tháng gần nhất tính tới đầu tháng sau.
3. Hệ thống truy vấn và trả về: giá trị đơn theo tháng, số tiền thực nhận theo tháng, tỉ lệ lấp đầy theo tháng, số đơn mới theo tháng, các loại phòng bán chạy nhất, tổng số đơn, số đơn huỷ, tỉ lệ huỷ, và số khoản chờ đối soát.
4. Quản trị viên xuất danh sách đơn ra tệp CSV nếu cần.

Luồng thay thế

- *2a. Quản trị viên chọn khoảng thời gian khác* → hệ thống chuẩn hoá về đầu tháng rồi tính lại.
- *3a. Không có đơn nào trong kỳ* → tỉ lệ huỷ trả về 0 thay vì gây lỗi chia cho không.

UC10 — Soạn nội dung trang chủ

Mục	Nội dung
Mã	UC10
Tác nhân chính	Quản trị viên
Mô tả	Cập nhật các khối nội dung trang chủ, biểu ngữ, thư viện ảnh và tin tức
Tiền điều kiện	Đã đăng nhập vai trò `ADMIN`
Hậu điều kiện	Nội dung đã lọc được lưu; trang công khai hiển thị nội dung mới

Luồng sự kiện chính

1. Quản trị viên mở một trong bốn màn hình quản lý nội dung.
2. Sửa nội dung, có thể nhập đoạn HTML cho phần thân.
3. Tải ảnh lên nếu cần.
4. Lưu. Hệ thống **lọc HTML *ở tầng vào*** theo danh sách thẻ cho phép tường minh trước khi ghi vào cơ sở dữ liệu.
5. Nội dung đã đăng xuất hiện trên trang công khai.

Luồng thay thế

- *4a. Nội dung chứa thẻ nguy hiểm* → các thẻ đó bị gỡ bỏ, phần còn lại vẫn lưu.
- *3a. Tệp tải lên sai định dạng hoặc vượt dung lượng* → từ chối với mã lỗi tương ứng.
- *5a. Bài viết chưa đánh dấu đã đăng* → không xuất hiện trên trang công khai.

UC11 — Viết đánh giá

Mục	Nội dung
Mã	UC11
Tác nhân chính	Khách đã trả phòng
Mô tả	Khách gửi đánh giá về kỳ lưu trú vừa qua
Tiền điều kiện	Đơn ở trạng thái `CHECKED_OUT`; khách có mã truy cập hoặc số điện thoại
Hậu điều kiện	Đánh giá được lưu ở trạng thái `PENDING`, chờ duyệt

Luồng sự kiện chính

1. Khách mở liên kết đánh giá và nhập điểm từ 1 tới 5, tiêu đề và nội dung.
2. Hệ thống xác thực khách qua mã truy cập hoặc số điện thoại.
3. Hệ thống kiểm tra đơn đã ở trạng thái `CHECKED_OUT`.
4. Hệ thống lưu đánh giá, **chụp tên khách từ đơn** chứ không nhận từ dữ liệu gửi lên, và đặt trạng thái chờ duyệt.
5. Quản trị viên duyệt, từ chối hoặc trả lời đánh giá.
6. Chỉ đánh giá đã duyệt mới hiển thị công khai.

Luồng thay thế

- *2a. Xác thực thất bại* → `BOOKING_NOT_FOUND`.
- *3a. Đơn chưa trả phòng* → từ chối kèm thông báo nêu rõ trạng thái hiện tại.

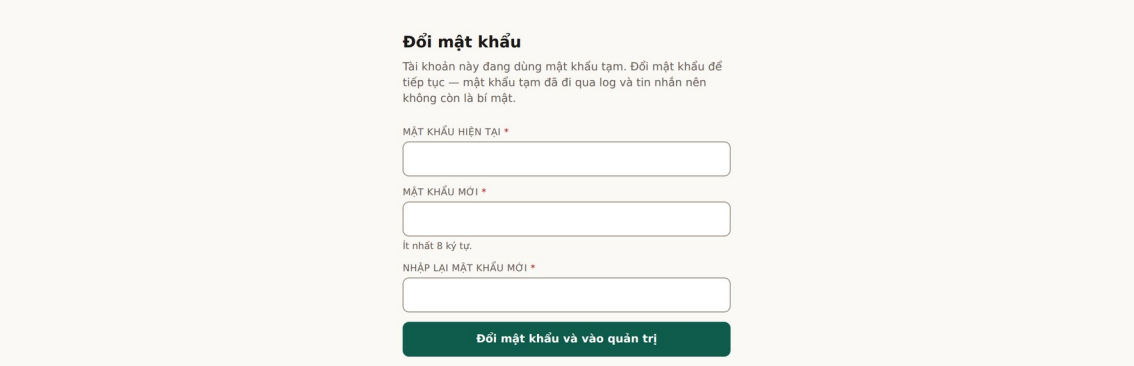
- *4a. Đơn đã có đánh giá* → từ chối. Cột khoá đơn có ràng buộc duy nhất nên cơ sở dữ liệu vẫn là chốt chặn cuối cùng; việc kiểm ở tầng dịch vụ chỉ để trả về một thông báo dễ hiểu thay vì lỗi ràng buộc thô.

Điểm thiết kế đáng lưu ý. Nội dung đánh giá do khách nhập được lưu **nguyên văn** và hiển thị dưới dạng văn bản thuần, không lọc HTML. Lý do: khách không có nhu cầu định dạng, nên cách an toàn nhất là không diễn giải nội dung đó như mã đánh dấu.

Ảnh màn hình bổ trợ cho phụ lục A

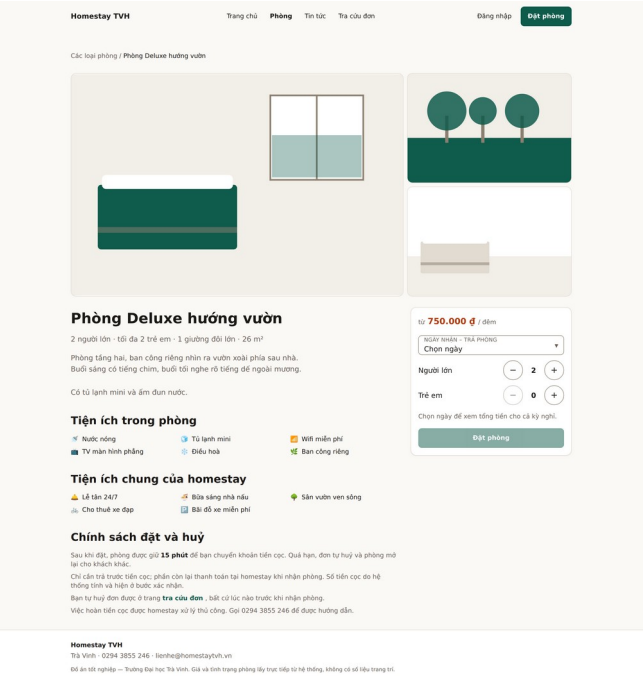
Hai màn hình dưới đây không nằm trong phần chữ của Chương 3 nhưng cần cho việc đối chiếu khi chấm: bước buộc đổi mật khẩu tạm ở lần đăng nhập quản trị đầu tiên, và trang chi tiết loại phòng mà khách thấy trước khi vào luồng đặt phòng.

Hình A.2 thể hiện bước buộc đổi mật khẩu tạm.



Hình A.2 — *Bước buộc đổi mật khẩu tạm (Nguồn: tác giả)*

Hình A.3 thể hiện trang chi tiết một loại phòng.



Hình A.3 — *Trang chi tiết một loại phòng (Nguồn: tác giả)*

Phụ lục B — Mô tả hai mươi mốt bảng

Danh sách cột đầy đủ của cả hai mươi mốt bảng nghiệp vụ, theo cùng mẫu ba cột tên cột – kiểu – mô tả và ràng buộc. Bốn bảng cốt lõi xếp lên đầu vì mục 3.2.2 giải thích vì sao mỗi ràng buộc của chúng tồn tại; mười bảy bảng còn lại xếp theo thứ tự migration.

Bảng `refresh_tokens` — phiên đăng nhập dài hạn

Tên cột	Kiểu	Mô tả và ràng buộc
`id`	bigserial	Khoá chính
`user_id`	bigint	Khoá ngoại tới `users`, xoá theo tầng
`token_hash`	char(64)	`uq_refresh_tokens_hash` duy nhất. **Lưu băm, không lưu token gốc**
`expires_at`	timestamptz	Thời điểm hết hạn
`revoked_at`	timestamptz	Thời điểm thu hồi, rỗng khi còn hiệu lực
`replaced_by`	char(64)	Băm của token kế nhiệm sau khi xoay vòng
`created_at`	timestamptz	Thời điểm tạo

Cột `replaced_by` phục vụ phát hiện token bị đánh cắp: nếu một token đã bị thay thế lại được sử dụng, đó là dấu hiệu bất thường và cả chuỗi token bị thu hồi.

Nhóm cốt lõi — bốn bảng mang ràng buộc quyết định

Bảng `users` — tài khoản người dùng

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
email	varchar(255)	uq_users_email duy nhất; ck_users_email_lower buộc lưu chữ thường
password_hash	varchar(72)	Băm mật khẩu theo thuật toán bcrypt
full_name	varchar(150)	Họ tên, bắt buộc
phone	varchar(20)	Số điện thoại, có thể rỗng
role	varchar(20)	ck_users_role giới hạn hai giá trị CUSTOMER và ADMIN
enabled	boolean	Mặc định bật; tắt để khoá tài khoản
must_change_password	boolean	Cờ buộc đổi mật khẩu tạm ở lần đăng nhập đầu
token_version	integer	ck_users_token_version không âm. Tăng khi đăng xuất hoặc đổi mật khẩu để vô hiệu hoá token cũ ngay
created_at, updated_at	timestamptz	Thời điểm tạo và cập nhật

Bảng `bookings` — đơn đặt phòng

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
code	varchar(20)	uq_bookings_code duy nhất. Mã đơn khách dùng để tra cứu
access_token	char(32)	uq_bookings_access_token duy nhất. **Mã truy cập — bí mật thao tác, khác mã đơn**
user_id	bigint	Khoá ngoại tới users, **rỗng với khách vãng lai**
guest_name	varchar(150)	Họ tên khách
guest_email	varchar(255)	Thư điện tử khách
guest_phone	varchar(20)	Số điện thoại khách
check_in, check_out	date	ck_bookings_dates buộc ngày trả sau ngày nhận
adults	integer	ck_bookings_adults tối thiểu 1
children	integer	ck_bookings_children không âm
room_type_id	bigint	Khoá ngoại tới room_types
room_type_name_snapshot	varchar(150)	Tên loại phòng chụp tại thời điểm đặt
unit_price_snapshot	numeric(12,2)	ck_bookings_unit_price lớn hơn 0. Giá chụp tại thời điểm đặt
room_quantity	integer	ck_bookings_room_qty tối thiểu 1
subtotal_amount	numeric(12,2)	ck_bookings_subtotal không âm
discount_amount	numeric(12,2)	ck_bookings_discount không âm

Tên cột	Kiểu	Mô tả và ràng buộc
total_amount	numeric(12,2)	ck_bookings_total không âm
deposit_amount	numeric(12,2)	ck_bookings_deposit không âm
promotion_id	bigint	Khoá ngoại fk_bookings_promotion thêm ở V5
status	varchar(20)	ck_bookings_status liệt kê **tám trạng thái**
payment_status	varchar(20)	ck_bookings_pay_status liệt kê bảy trạng thái thanh toán
special_request	text	Yêu cầu đặc biệt của khách
hold_expires_at	timestamptz	Hạn giữ chỗ
client_ip	inet	Địa chỉ mạng của người đặt
user_agent	varchar(255)	Thông tin trình duyệt
cancelled_at, cancel_reason	timestamptz, text	Thời điểm và lý do huỷ
created_at, updated_at	timestamptz	Thời điểm tạo và cập nhật

Bảng `booking_rooms` — gán phòng vật lý cho đơn

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
booking_id	bigint	Khoá ngoại tới bookings, xoá theo tầng
room_id	bigint	Khoá ngoại tới rooms
check_in, check_out	date	ck_booking_rooms_dates buộc ngày trả sau ngày nhận
stay	daterange	**Cột sinh tự động** GENERATED ALWAYS AS (daterange(check_in, check_out, '[)')) STORED
status	varchar(20)	ck_booking_rooms_status giới hạn ACTIVE và RELEASED

Bảng `room_closures` — khoảng ngày phòng không nhận khách

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
room_id	bigint	Khoá ngoại tới rooms, xoá theo tầng
from_date	date	Đêm đầu tiên bị chặn
to_date	date	Ngày mở bán lại. ck_room_closures_dates buộc lớn hơn from_date
blocked	daterange	**Cột sinh tự động** theo quy ước nửa mở, cùng quy ước với booking_rooms.stay
reason	varchar(300)	Lý do đóng phòng
created_by	bigint	Khoá ngoại tới users, đặt rỗng khi tài khoản bị xoá
created_at	timestamptz	Thời điểm tạo

Nhóm V2 — Phòng và tiện nghi

Bảng `amenities` — tiện nghi

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
code	varchar(50)	uq_amenities_code duy nhất
name	varchar(100)	Tên hiển thị
icon	varchar(50)	Mã biểu tượng
category	varchar(20)	ck_amenities_category giới hạn ROOM và PROPERTY
display_order	integer	Thứ tự hiển thị

Bảng `room_types` — loại phòng

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
code	varchar(50)	uq_room_types_code duy nhất
slug	varchar(120)	uq_room_types_slug duy nhất, dùng trong đường dẫn
name	varchar(150)	Tên loại phòng
short_description, description	text	Mô tả ngắn và mô tả đầy đủ
base_price	numeric(12,2)	ck_room_types_price phải lớn hơn 0
capacity_adults	integer	ck_room_types_adults tối thiểu 1
capacity_children	integer	ck_room_types_children không âm
bed_info	varchar(150)	Mô tả giường
area_sqm	numeric(6,2)	ck_room_types_area rỗng hoặc lớn hơn 0
display_order	integer	Thứ tự hiển thị
active	boolean	Tắt để ngừng bán loại phòng
created_at, updated_at	timestamptz	Thời điểm tạo và cập nhật

Bảng `rooms` — phòng vật lý

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
room_type_id	bigint	Khoá ngoại tới room_types
room_number	varchar(20)	uq_rooms_number duy nhất
floor	integer	Tầng, có thể rỗng
status	varchar(20)	ck_rooms_status giới hạn AVAILABLE, MAINTENANCE, OUT_OF_SERVICE
note	text	Ghi chú nội bộ
created_at, updated_at	timestamptz	Thời điểm tạo và cập nhật

Đây là bảng mà ràng buộc chống trùng lịch đặt lên. Chỉ mục `idx_rooms_type_status` phục vụ truy vấn đếm phòng trống theo loại.

Bảng `room_type_images` — ảnh của loại phòng

Tên cột	Kiểu	Mô tả và ràng buộc
`id`	bigserial	Khoá chính
`room_type_id`	bigint	Khoá ngoại, xoá theo tầng
`url`	varchar(500)	Đường dẫn ảnh
`public_id`	varchar(255)	Định danh trên dịch vụ lưu ảnh, rỗng khi lưu cục bộ
`alt_text`	varchar(255)	Văn bản thay thế phục vụ khả năng tiếp cận
`display_order`	integer	Thứ tự hiển thị
`is_cover`	boolean	Ảnh bìa. Chỉ mục duy nhất từng phần `uq_room_type_images_cover` bảo đảm mỗi loại phòng có tối đa một ảnh bìa

Bảng `room_type_amenities` — bảng nối loại phòng và tiện nghi

Tên cột	Kiểu	Mô tả và ràng buộc
`room_type_id`	bigint	Khoá ngoại, xoá theo tầng
`amenity_id`	bigint	Khoá ngoại, xoá theo tầng
		`pk_room_type_amenities` khoá chính tổ hợp hai cột

Đây là bảng duy nhất trong lược đồ **không có thực thể riêng** trong mã nguồn. Quan hệ nhiều–nhiều giữa loại phòng và tiện nghi được ánh xạ trực tiếp, nên tổng số thực thể là **20** trong khi tổng số bảng là **21**.

Nhóm V3 — Đặt phòng

Bảng `booking_status_history` — nhật ký chuyển trạng thái

Tên cột	Kiểu	Mô tả và ràng buộc
`id`	bigserial	Khoá chính
`booking_id`	bigint	Khoá ngoại, xoá theo tầng
`from_status`	varchar(20)	`ck_bsh_from_status` rỗng hoặc thuộc tám trạng thái
`to_status`	varchar(20)	`ck_bsh_to_status` thuộc tám trạng thái
`changed_by`	bigint	Khoá ngoại tới `users`, rỗng khi hệ thống tự chuyển
`actor`	varchar(20)	`ck_bsh_actor` giới hạn `GUEST`, `CUSTOMER`, `ADMIN`, `SYSTEM`
`note`	text	Ghi chú kèm theo lần chuyển
`created_at`	timestamptz	Thời điểm chuyển

Nhóm V4 — Thanh toán và thư

Bảng `payments` — lần thanh toán

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
booking_id	bigint	Khoá ngoại, xoá theo tầng
attempt_no	integer	ck_payments_attempt tối thiểu 1. Cùng booking_id tạo thành chỉ mục duy nhất
provider	varchar(20)	ck_payments_provider giới hạn SEPAY và MANUAL
amount_expected	numeric(12,2)	ck_payments_expected lớn hơn 0
amount_received	numeric(12,2)	ck_payments_received không âm
qr_content, qr_image_url	text, varchar(500)	Nội dung và ảnh mã QR
transfer_content	varchar(50)	uq_payments_transfer_content duy nhất. Mã đơn ghép số thứ tự lần thử
status	varchar(20)	ck_payments_status liệt kê sáu trạng thái
reconcile_status	varchar(20)	ck_payments_reconcile giới hạn NONE, NEEDS_REVIEW, REFUND_REQUIRED, RESOLVED
provider_txn_id	varchar(100)	Mã giao dịch của nhà cung cấp
paid_at, expires_at	timestamptz	Thời điểm thanh toán và hết hạn
created_at, updated_at	timestamptz	Thời điểm tạo và cập nhật

Chỉ mục từng phần idx_payments_reconcile chỉ đánh chỉ mục các dòng cần đối soát, giúp màn hình đối soát truy vấn nhanh mà không phải quét toàn bảng.

Bảng `payment_webhook_events` — nhật ký webhook

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
provider	varchar(20)	Nhà cung cấp gửi webhook
external_id	varchar(100)	Mã sự kiện bên nhà cung cấp
payment_id	bigint	Khoá ngoại tới payments, rỗng khi không khớp đơn nào
payload	jsonb	Toàn bộ nội dung nhận được
processing_result	varchar(20)	ck_pwe_result giới hạn MATCHED, UNMATCHED, LATE, DUPLICATE, ERROR
error_message	text	Thông báo lỗi nếu xử lý thất bại
received_at, processed_at	timestamptz	Thời điểm nhận và xử lý

Chỉ mục duy nhất uq_webhook_provider_external trên cặp nhà cung cấp và mã sự kiện là cơ chế **chống cộng tiền hai lần** khi nhà cung cấp gửi lại webhook.

Bảng `outbound_emails` — hàng đợi thư đi

Tên cột	Kiểu	Mô tả và ràng buộc
`id`	bigserial	Khoá chính
`booking_id`	bigint	Khoá ngoại, đặt rỗng khi đơn bị xoá
`template`	varchar(100)	Tên mẫu thư
`to_email`	varchar(255)	Địa chỉ nhận
`payload`	jsonb	Dữ liệu điền vào mẫu
`status`	varchar(20)	`ck_outbound_emails_status` giới hạn `PENDING`, `SENT`, `FAILED`
`attempts`	integer	`ck_outbound_emails_attempts` không âm
`last_error`	text	Lỗi lần gửi gần nhất
`sent_at`, `created_at`	timestamptz	Thời điểm gửi và tạo

Nhóm V5 — Khuyến mãi và đánh giá

Bảng `promotions` — mã khuyến mãi

Tên cột	Kiểu	Mô tả và ràng buộc
`id`	bigserial	Khoá chính
`code`	varchar(50)	`uq_promotions_code` duy nhất
`name`, `description`	varchar(150), text	Tên và mô tả
`discount_type`	varchar(20)	`ck_promotions_type` giới hạn `PERCENT` và `FIXED`
`discount_value`	numeric(12,2)	`ck_promotions_value` lớn hơn 0
`max_discount_amount`	numeric(12,2)	`ck_promotions_max` rỗng hoặc lớn hơn 0
`min_nights`	integer	`ck_promotions_nights` tối thiểu 1
`min_total_amount`	numeric(12,2)	`ck_promotions_min_total` không âm
`starts_at`, `ends_at`	timestamptz	`ck_promotions_window` buộc kết thúc sau bắt đầu
`usage_limit`	integer	Giới hạn lượt dùng, rỗng là không giới hạn
`used_count`	integer	`ck_promotions_used` không âm; `ck_promotions_usage` buộc không vượt giới hạn
`active`	boolean	Tắt để ngừng áp dụng

Ràng buộc `ck_promotions_usage` bảo đảm số lượt đã dùng **không bao giờ vượt quá giới hạn** ngay ở tầng dữ liệu, kể cả khi nhiều đơn cùng áp một mã.

Bảng `reviews` — đánh giá của khách

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
booking_id	bigint	Khoá ngoại, xoá theo tầng. uq_reviews_booking **duy nhất — mỗi đơn một đánh giá**
user_id	bigint	Khoá ngoại tới users, rỗng với khách vãng lai
guest_name_snapshot	varchar(150)	Tên khách chụp từ đơn tại thời điểm đánh giá
rating	smallint	ck_reviews_rating trong khoảng 1 tới 5
title, content	varchar(200), text	Tiêu đề và nội dung
status	varchar(20)	ck_reviews_status giới hạn PENDING, APPROVED, REJECTED
admin_reply, replied_at	text, timestamptz	Phản hồi của quản trị viên
created_at	timestamptz	Thời điểm gửi

Nhóm V6 — Nội dung

Bảng `site_contents` — khối nội dung trang chủ

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
section_key	varchar(100)	uq_site_contents_key duy nhất. Khoá định danh khối
title, subtitle	varchar(255)	Tiêu đề và tiêu đề phụ
body	text	Nội dung HTML **đã lọc ở tầng vào**
data	jsonb	Dữ liệu có cấu trúc tuỳ khối
image_url	varchar(500)	Ảnh minh hoạ
updated_by	bigint	Khoá ngoại tới users
updated_at	timestamptz	Thời điểm cập nhật

Bảng `banners` — biểu ngữ

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
title	varchar(255)	Tiêu đề, bắt buộc
image_url	varchar(500)	Ảnh, bắt buộc
public_id	varchar(255)	Định danh trên dịch vụ lưu ảnh
link_url	varchar(500)	Đường dẫn khi nhấp vào
display_order	integer	Thứ tự hiển thị
active	boolean	Bật tắt hiển thị
starts_at, ends_at	timestamptz	ck_banners_window buộc kết thúc sau bắt đầu khi cả hai cùng có giá trị

Bảng `gallery_images` — thư viện ảnh

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
url	varchar(500)	Đường dẫn ảnh, bắt buộc
public_id	varchar(255)	Định danh trên dịch vụ lưu ảnh
caption	varchar(255)	Chú thích
category	varchar(50)	Phân nhóm ảnh
display_order	integer	Thứ tự hiển thị
active	boolean	Bật tắt hiển thị

Bảng `posts` — tin tức

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
slug	varchar(200)	uq_posts_slug duy nhất, dùng trong đường dẫn
title	varchar(255)	Tiêu đề, bắt buộc
excerpt, content	text	Trích dẫn ngắn và nội dung đầy đủ
cover_image_url	varchar(500)	Ảnh bìa
published	boolean	Cờ đã đăng
published_at	timestamptz	Thời điểm đăng
author_id	bigint	Khoá ngoại tới users

Chỉ mục từng phần idx_posts_published chỉ đánh chỉ mục các bài đã đăng, phục vụ trang tin tức công khai.

Nhóm V7 — Ghi chú nội bộ

Bảng `booking_notes` — ghi chú của quản trị viên trên đơn

Tên cột	Kiểu	Mô tả và ràng buộc
id	bigserial	Khoá chính
booking_id	bigint	Khoá ngoại, xoá theo tầng
author_id	bigint	Khoá ngoại tới users, đặt rỗng khi tài khoản bị xoá
content	text	ck_booking_notes_content buộc không rỗng sau khi cắt khoảng trắng
created_at	timestamptz	Thời điểm ghi

Bảng này tách khỏi booking_status_history vì mỗi dòng lịch sử bắt buộc kèm một lần chuyển trạng thái thật, mà máy trạng thái không cho phép chuyển từ một trạng thái sang chính nó. Ghi chú là dữ liệu **nội bộ** và không endpoint công khai nào đọc bảng này.

Nhóm V8 — Ngày khả dụng

PHỤ LỤC (TIẾP THEO)

Phụ lục C — Mã nguồn migration tạo ràng buộc chống trùng lịch

Trích từ tệp `V3__bookings_and_exclusion.sql`:

```
CREATE TABLE booking_rooms (
    id         bigserial   PRIMARY KEY,
    booking_id bigint      NOT NULL REFERENCES bookings (id) ON
DELETE CASCADE,
    room_id    bigint      NOT NULL REFERENCES rooms (id),
    check_in   date        NOT NULL,
    check_out  date        NOT NULL,
    stay       daterange   GENERATED ALWAYS AS
                           (daterange(check_in, check_out, '[)'))
STORED,
    status     varchar(20) NOT NULL DEFAULT 'ACTIVE',
    CONSTRAINT ck_booking_rooms_dates  CHECK (check_out > check_in),
    CONSTRAINT ck_booking_rooms_status CHECK (status IN ('ACTIVE',
'RELEASED'))
);

ALTER TABLE booking_rooms
    ADD CONSTRAINT booking_rooms_no_overlap
    EXCLUDE USING gist (room_id WITH =, stay WITH &&)
    WHERE (status = 'ACTIVE');
```

Phụ lục D — Trigger ràng buộc hoãn

Trích từ cùng tệp migration:

```
CREATE FUNCTION assert_booking_room_count(p_booking_id bigint)
RETURNS void AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM bookings b
        WHERE b.id = p_booking_id
          AND b.status NOT IN ('CANCELLED', 'EXPIRED', 'NO_SHOW')
          AND b.room_quantity <> (
              SELECT count(*)
              FROM booking_rooms br
              WHERE br.booking_id = b.id
                AND br.status = 'ACTIVE')
    ) THEN
        RAISE EXCEPTION 'booking_rooms khong khop room_quantity';
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER booking_room_count_check
    AFTER INSERT OR UPDATE OR DELETE ON booking_rooms
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION check_booking_room_count();
```

Trigger được đặt ở **cả hai phía**: khi sửa bảng gán phòng, và khi sửa số lượng phòng trên đơn. Đặt một phía là để lọt trường hợp còn lại.

Phụ lục E — Bảng endpoint đầy đủ

Bảng đầy đủ 82 thao tác trên 65 đường dẫn được liệt kê trong tài liệu kỹ thuật `docs/api.md` của dự án, và xem được trực tiếp qua giao diện tài liệu tương tác của hệ thống đang chạy ở cấu hình trình diễn.

Cách đếm lại số thao tác từ hệ thống đang chạy:

```
curl -s localhost/v3/api-docs | python3 -c "
import json,sys
d = json.load(sys.stdin)
methods = ('get','post','put','delete','patch')
print(sum(len([m for m in v if m in methods]) for v in
d['paths'].values()))"
```

Phụ lục F — Migration bảng khoảng đóng phòng

Trích từ tệp `V8__room_closures.sql`:

```
CREATE TABLE room_closures (
    id         bigserial   PRIMARY KEY,
    room_id    bigint      NOT NULL REFERENCES rooms (id) ON DELETE
CASCADE,
    from_date  date        NOT NULL,
    to_date    date        NOT NULL,
    blocked    daterange   GENERATED ALWAYS AS
                           (daterange(from_date, to_date, '[)'))
STORED,
    reason     varchar(300),
    created_by bigint      REFERENCES users (id) ON DELETE SET NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT ck_room_closures_dates CHECK (to_date > from_date)
);

ALTER TABLE room_closures
    ADD CONSTRAINT room_closures_no_overlap
    EXCLUDE USING gist (room_id WITH =, blocked WITH &&);

CREATE INDEX idx_room_closures_room ON room_closures (room_id,
from_date);
```

Phụ lục G — Cấu hình đóng gói

Tệp cấu hình đóng gói `docker-compose.yml` và cấu hình máy chủ web `frontend/nginx.conf` được lưu trong kho mã nguồn của dự án.

Điểm đáng lưu ý trong cấu hình máy chủ web là cú pháp tiền tố ưu tiên ở cả bốn khối chuyển tiếp:

```
location ^~ /api/         { ... }
location ^~ /swagger-ui/  { ... }
location ^~ /v3/api-docs/ { ... }
location ^~ /uploads/     { ... }
```

Phụ lục H — Lệnh kiểm chứng

Mục đích	Lệnh
Chạy toàn bộ kiểm thử	`cd backend && ./mvnw verify`
Biên dịch tầng giao diện	`cd frontend && npm run build`
Kiểm tra quy ước mã nguồn	`cd frontend && npm run lint`
Dựng và chạy toàn hệ thống	`docker compose up -d --build`
Kiểm tra trạng thái các dịch vụ	`docker compose ps`
Đếm số bảng trong lược đồ	`SELECT count(*) FROM information_schema.tables WHERE table_schema = 'public' AND table_name <> 'flyway_schema_history';`

Phụ lục I — Chi tiết 19 lớp kiểm thử

4.2.1. Bảng tổng hợp theo lớp

Lớp kiểm thử	Số ca	Chứng minh điều gì
`SchemaMigrationTest`	8	Migration dựng đúng **21 bảng**; mọi migration thành công; không migration nào bị sửa sau khi phát hành; cả hai cột kiểu khoảng đều là cột sinh tự động
`SchemaConstraintIT`	9	Từng ràng buộc kiểm tra thật sự chặn: thư điện tử không chữ thường, trạng thái sai chính tả, số tiền âm, ngày trả không sau ngày nhận, điểm đánh giá ngoài khoảng 1 tới 5
`SqlStatesIT`	2	Hai mã lỗi `23P01` và `25P02` được nhận diện đúng
`BookingConcurrencyIT`	3	**Lớp quan trọng nhất.** Nhiều luồng thật cùng đặt phòng cuối — đúng một luồng thắng; sau khi bắt lỗi ràng buộc, vòng thử tiếp theo phải ở giao dịch mới; đơn nhiều phòng gán được nhờ trigger hoãn
`AvailabilityQueryIT`	6	Đếm phòng trống đúng trong các tình huống dễ đếm sai: phòng bảo trì đang có đơn không bị trừ hai lần; loại phòng hết sạch biến mất khỏi kết quả; khoảng nửa mở cho khách trả và khách nhận cùng ngày; sức chứa so theo từng phòng; lịch trả đủ mọi ngày
`BookingLifecycleIT`	8	Bảng chuyển trạng thái: bước

Lớp kiểm thử	Số ca	Chứng minh điều gì
		hợp lệ đi được, bước không hợp lệ bị từ chối. `CHECKED_OUT` không nhả phòng
`BookingExpiryIT`	4	Bộ quét chuyển đơn quá hạn, nhả phòng, hoàn lượt khuyến mãi; đơn đã có tiền không bị quét
`RoomClosureIT`	9	Khoảng đóng phòng trừ đúng phòng, đúng đêm, ở cả bốn truy vấn; biên nửa mở; lịch không rơi mất ngày; khoảng chồng nhau bị từ chối; đóng phòng không huỷ đơn; khoảng đã qua rời khỏi danh sách thao tác
`SepayWebhookIT`	10	Sai khoá xác thực bị từ chối; webhook gửi lại không cộng tiền hai lần; đủ tiền thì xác nhận; thiếu tiền thì chờ đối soát; thừa tiền thì đánh dấu hoàn; webhook không khớp đơn nào vẫn được ghi nhật ký
`PaymentRaceIT`	5	Cửa sổ tranh chấp giữa webhook và bộ quét hết hạn; nhánh giành lại phòng tôn trọng khoảng đóng phòng
`AuthFlowIT`	7	Đăng ký, đăng nhập, làm mới token, đăng xuất; số phiên bản token tăng làm token cũ mất hiệu lực ngay; token làm mới lưu dạng băm
`EndpointAuthorizationIT`	3	Mọi endpoint đã đăng ký nằm trong ma trận 30 tiền tố; cấm khai dòng bao trùm; endpoint không khai gì bị chặn
`ContentSanitizerTest`	17	Bộ lọc HTML gỡ thẻ kịch bản, chặn giao thức nguy hiểm trong liên kết, gỡ khung nhúng, gỡ ảnh ngoài danh sách nguồn cho phép, giữ thẻ hợp lệ
`PublicContentIT`	7	Giao diện công khai chỉ trả nội dung đã đăng; nội dung mặc định hiện ra khi chưa soạn; bài viết nháp không lộ
`ReviewIT`	8	Gửi đánh giá cần xác thực; tên khách chụp từ đơn, không nhận từ dữ liệu gửi lên; ràng buộc duy nhất chặn đánh giá thứ hai; chỉ đánh giá đã duyệt mới công khai
`DashboardServiceIT`	6	Doanh thu theo tháng, tỉ lệ lấp đầy, tỉ lệ huỷ tính đúng trên dữ liệu thật
`CsvExportTest`	10	Xuất tệp CSV thoát đúng dấu

Lớp kiểm thử	Số ca	Chứng minh điều gì
		phẩy, dấu nháy và ký tự xuống dòng trong dữ liệu
`ImageUploadIT`	7	Từ chối kiểu tệp sai; từ chối tệp vượt dung lượng; giải mã lại ảnh và đổi tên; chỉ quản trị viên tải lên được
Tổng	**129**	

Phụ lục J — Bảng đầy đủ mã lỗi

3.5.4. Thiết kế mã lỗi

Phản hồi lỗi theo chuẩn RFC 7807 [4] kèm trường `code` tự thêm. Tầng giao diện đọc trường `code`, **không** đọc câu chữ mô tả, nên việc sửa câu chữ ở tầng máy chủ không làm hỏng màn hình.

Mã lỗi	HTTP	Ý nghĩa
`ROOM_NOT_AVAILABLE`	409	Phòng vừa được đặt hết cho khoảng ngày này
`INVALID_PROMOTION`	400	Mã không hợp lệ hoặc chưa đủ điều kiện
`PROMOTION_EXHAUSTED`	409	Mã đã hết lượt
`BOOKING_NOT_FOUND`	404	Không có đơn khớp mã và số điện thoại
`INVALID_ACCESS_TOKEN`	403	Mã truy cập sai
`INVALID_STATE_TRANSITION`	409	Bước chuyển trạng thái không hợp lệ
`INVALID_BOOKING_REQUEST`	400	Dữ liệu đơn không hợp lệ
`INVALID_ADMIN_REQUEST`	400	Dữ liệu quản trị không hợp lệ
`CLOSURE_OVERLAP`	409	Khoảng đóng phòng chồng lên khoảng đã có
`RESOURCE_IN_USE`	409	Còn dữ liệu tham chiếu tới bản ghi cần xoá
`PASSWORD_CHANGE_REQUIRED`	403	Phải đổi mật khẩu tạm trước
`UNSUPPORTED_IMAGE_TYPE`	415	Định dạng ảnh không được chấp nhận
`IMAGE_TOO_LARGE`	413	Ảnh vượt giới hạn dung lượng
`TOO_MANY_REQUESTS`	429	Vượt giới hạn tần suất
`UNAUTHORIZED`	401	Thiếu hoặc sai token

Phụ lục K — Danh sách 32 yêu cầu chức năng

1.4.1. Yêu cầu chức năng

Nhóm A — Dành cho khách

Mã	Yêu cầu
CN-01	Xem danh sách loại phòng kèm tiện nghi, hình

Mã	Yêu cầu
	ảnh, giá
CN-02	Xem chi tiết một loại phòng, xem thư viện ảnh
CN-03	Tìm phòng trống theo khoảng ngày, số người lớn, trẻ em, số phòng
CN-04	Xem lịch từng đêm: đêm nào còn phòng, đêm nào hết
CN-05	Gửi yêu cầu đặt phòng, nhận mã đơn
CN-06	Xem chi phí dự kiến trước khi xác nhận đơn
CN-07	Áp dụng mã khuyến mãi và thấy ngay số tiền được giảm
CN-08	Thanh toán đặt cọc bằng mã QR
CN-09	Tra cứu đơn bằng mã đơn và số điện thoại
CN-10	Huỷ đơn
CN-11	Nhận thư xác nhận qua thư điện tử
CN-12	Viết đánh giá sau khi trả phòng
CN-13	Đăng ký, đăng nhập, xem lại toàn bộ đơn đã đặt
CN-14	Đọc tin tức, xem thư viện ảnh của homestay

Nhóm B — Dành cho quản trị viên

Mã	Yêu cầu
CN-15	Đăng nhập khu quản trị; buộc đổi mật khẩu tạm ở lần đầu
CN-16	Xem danh sách đơn, lọc theo trạng thái và khoảng ngày
CN-17	Xem chi tiết đơn: lịch sử trạng thái, thanh toán, thư đã gửi, ghi chú
CN-18	Chuyển trạng thái đơn theo máy trạng thái hợp lệ
CN-19	Ghi chú nội bộ trên đơn
CN-20	Đối soát thanh toán: xác nhận thủ công, xử lý khoản lệch
CN-21	Quản lý loại phòng: thêm, sửa, xoá, gán tiện nghi, tải ảnh
CN-22	Quản lý phòng vật lý: thêm, sửa, đổi trạng thái vận hành
CN-23	**Đóng phòng theo khoảng ngày** — quản lý ngày khả dụng
CN-24	Quản lý mã khuyến mãi
CN-25	Duyệt, từ chối, trả lời đánh giá của khách
CN-26	Quản lý nội dung trang chủ, biểu ngữ, thư viện ảnh, tin tức
CN-27	Xem tổng quan: doanh thu, tỉ lệ lấp đầy, tỉ lệ huỷ
CN-28	Xuất báo cáo đơn ra tệp định dạng CSV

Nhóm C — Hệ thống tự động

Mã	Yêu cầu
CN-29	Nhận webhook khi có tiền về, tự đối chiếu và xác nhận đơn

Mã	Yêu cầu
CN-30	Tự chuyển đơn quá hạn giữ chỗ sang trạng thái hết hạn và nhả phòng
CN-31	Gửi thư qua hàng đợi, ghi nhận mọi thư đã gửi
CN-32	Xử lý tiền về muộn: mở lại đơn, thử giành lại phòng

Phụ lục L — Bảng tổng hợp công nghệ

Tầng	Công nghệ	Phiên bản
Ngôn ngữ tầng máy chủ	Java	21 (LTS)
Nền tảng tầng máy chủ	Spring Boot	3.5.6
Truy cập dữ liệu	Spring Data JPA / Hibernate, Spring JDBC	theo nền tảng
Bảo mật	Spring Security và JWT	theo nền tảng
Cơ sở dữ liệu	PostgreSQL	16
Quản lý lược đồ	Flyway	theo nền tảng
Giới hạn tần suất	bucket4j	—
Lọc HTML	OWASP java-html-sanitizer	20240325.1
Kiểm thử	JUnit 5, AssertJ, Testcontainers	—
Ngôn ngữ tầng giao diện	TypeScript	theo Angular
Nền tảng tầng giao diện	Angular	21
Định kiểu	Tailwind CSS	4
Máy chủ web	nginx	alpine
Đóng gói	Docker, Docker Compose	Engine 24+, Compose v2
Thư trong môi trường trình diễn	Mailpit	—
Thanh toán	SePay và VietQR	—

Phụ lục M — Chi tiết kiểm thử bảo mật

Kịch bản tấn công	Kết quả
Giả mạo tiêu đề địa chỉ để vượt giới hạn tần suất	**Thất bại** — vẫn bị chặn ở đúng ngưỡng
Gọi endpoint quản trị không có token	Từ chối, mã 401
Gọi endpoint quản trị bằng token vai trò khách	Từ chối, mã 403
Dùng token cũ sau khi đã đổi mật khẩu	Từ chối, mã 401 — token cũ mất hiệu lực ngay
Gọi webhook không có khoá xác thực	Từ chối
Gửi lại cùng một webhook lần hai	Bỏ qua, không cộng tiền hai lần
Chèn thẻ kịch bản vào nội dung quản lý	Bị bộ lọc gỡ bỏ
Xem trạng thái thanh toán chỉ bằng mã đơn	Từ chối — bắt buộc mã truy cập
Truy cập đơn của người khác qua tham số đường dẫn	Từ chối — lọc theo token của phiên

Phụ lục N — Danh sách kiểm thử thủ công

Danh sách tối thiểu cần chạy trên bản đóng gói:

1. Trang chủ có nội dung thật: 4 loại phòng, thư viện ảnh, đánh giá, tin tức.
2. Đặt một đơn từ đầu đến khi ra mã QR.

3. Tải lại trang ở màn hình thanh toán — vẫn thấy mã QR và đồng hồ đếm ngược.
4. Tra cứu đơn bằng mã và số điện thoại; thử sai số điện thoại thì bị từ chối.
5. Huỷ đơn; kiểm tra phòng mở lại bằng cách đặt lại đúng ngày đó.
6. Đăng nhập quản trị, bị buộc đổi mật khẩu, rồi vào trang tổng quan.
7. Màn hình đối soát có sẵn khoản cần xử lý.
8. Sửa một khối nội dung trang chủ rồi mở lại trang chủ để thấy thay đổi.
9. Mở hộp thư giả lập xem thư xác nhận đã gửi.
10. Đóng một phòng vài ngày; tìm phòng đúng khoảng đó thấy số phòng giảm đúng một, và đêm mở bán lại **không** giảm.

Phụ lục O — Mười hai yêu cầu phi chức năng

Bảng đầy đủ của mục 1.4.2. Cột cuối là cách kiểm chứng, không phải lời hứa.

Mã	Loại	Yêu cầu	Cách kiểm chứng
PCN-01	Toàn vẹn	Không bao giờ bán trùng một phòng, kể cả khi có tranh chấp	Kiểm thử đa luồng thật
PCN-02	Toàn vẹn	Số phòng đã gán luôn khớp số phòng đơn yêu cầu	Trigger hoãn ở cơ sở dữ liệu
PCN-03	Toàn vẹn	Không nhánh nào để tiền biến mất im lặng	Kiểm thử cửa sổ tranh chấp webhook
PCN-04	Bảo mật	Mặc định từ chối mọi endpoint chưa khai quyền	Kiểm thử ma trận 30 tiền tố
PCN-05	Bảo mật	Chống dò mật khẩu và spam đặt phòng	Giới hạn tần suất theo 9 khoá
PCN-06	Bảo mật	Không có bí mật nào nằm trong mã nguồn	Ứng dụng dừng khởi động khi thiếu biến
PCN-07	Khả dụng	Giao diện sử dụng được trên điện thoại	Kiểm ở 3 độ rộng màn hình
PCN-08	Khả dụng	Đạt chuẩn WCAG 2.1 mức A và AA [13]	Quét tự động
PCN-09	Khả dụng	Mọi vùng chạm tối thiểu 44×44 điểm ảnh	Đo tự động
PCN-10	Hiệu năng	Lịch cả kỳ lấy trong một truy vấn, không lặp từng đêm	Đọc mã truy vấn
PCN-11	Triển khai	Ba lệnh từ lúc sao chép mã nguồn tới lúc chạy	Thực nghiệm
PCN-12	Bảo trì	Lược đồ do công cụ migration quản lý, migration bất biến	Kiểm thử checksum

Phụ lục P — Cây gói tầng máy chủ

Cây gói đầy đủ của mục 3.4.2.

```
com.tvh.homestay
├── auth/           – đăng nhập, token, phiên làm việc
├── user/           – người dùng
├── room/           – loại phòng, phòng, tiện nghi, khoảng đóng
├── availability/   – truy vấn phòng trống
├── booking/        – đơn, máy trạng thái, gán phòng, tính giá
├── payment/        – webhook, đối soát
├── promotion/      – mã khuyến mãi
├── review/         – đánh giá
├── cms/            – thực thể nội dung trang chủ
├── content/        – điều khiển nội dung công khai và quản trị
├── admin/          – điều khiển và dịch vụ khu quản trị
├── report/         – tổng quan, xuất tệp CSV
├── storage/        – tải ảnh lên
├── mail/           – hàng đợi thư
├── common/         – xử lý ngoại lệ, tiện ích dùng chung
└── demo/           – nạp dữ liệu mẫu, chỉ hoạt động ở cấu hình trình
diễn
```

Phụ lục Q — Chín khoá giới hạn tần suất

Hạn mức đầy đủ của mục 3.5.3.

Khoá	Hạn mức	Áp dụng cho
`ip:auth`	10 lần mỗi phút	Mọi endpoint xác thực
`email:login`	5 lần mỗi phút	Đăng nhập, theo thư điện tử trong thân yêu cầu
`ip:availability`	60 lần mỗi phút	Truy vấn phòng trống
`ip:promo`	20 lần mỗi phút	Kiểm tra mã khuyến mãi
`ip:booking-create`	10 lần mỗi phút	Tạo đơn
`phone:booking`	10 lần mỗi giờ	Tạo đơn, theo số điện thoại
`ip:lookup`	10 lần mỗi phút	Tra cứu đơn
`code:lookup`	5 lần mỗi giờ	Tra cứu đơn, theo mã đơn
`ip:cancel`	10 lần mỗi phút	Huỷ đơn

Phụ lục R — Nhật ký mười một lỗi và cách khắc phục

Bảng đầy đủ bốn cột của mục 4.3. Phần thân bài giữ hiện tượng và nguyên nhân gốc vì đó là phần có giá trị rút kinh nghiệm; cách khắc phục cụ thể của từng lỗi là thông tin tra cứu nên đặt ở đây.

#	Hiện tượng	Nguyên nhân gốc	Cách khắc phục
1	Giao dịch thử phòng tiếp theo luôn thất bại	Bắt lỗi ràng buộc rồi thử tiếp **trong cùng giao dịch**; PostgreSQL đã huỷ giao dịch nên mọi lệnh sau trả `25P02`	Mỗi lượt thử chạy trong một giao dịch mới
2	Website báo thiếu	Truy vấn lấy tổng trừ	Đếm trực tiếp phòng

#	Hiện tượng	Nguyên nhân gốc	Cách khắc phục
	phòng so với thực tế	đi số đã đặt trừ **hai lần** phòng bảo trì đang có đơn	vừa khả dụng vừa rảnh
3	Giao dịch bị bác khi khách trả phòng	Nhả phòng lúc chuyển sang `CHECKED_OUT` làm trigger kiểm số phòng bác cả giao dịch	Trạng thái `CHECKED_OUT` không nhả phòng
4	**Toàn bộ trang web hiển thị không có định kiểu**	Chính sách bảo mật nội dung chặn thuộc tính sự kiện mà công cụ tối ưu định kiểu tự sinh ra	Tắt tính năng tối ưu đó thay vì nới lỏng chính sách bảo mật
5	**Mọi nút bấm chính có độ tương phản khoảng 2:1**	Tệp định kiểu nền **không nằm trong tầng CSS** nên thắng mọi lớp tiện ích bất kể độ ưu tiên	Bọc tệp nền trong tầng cơ sở
6	Ảnh tải lên trả về lỗi 404	Khối biểu thức chính quy của máy chủ web thắng khối tiền tố thường	Dùng cú pháp tiền tố ưu tiên cho cả bốn khối chuyển tiếp
7	Màn hình thanh toán trắng trơn	Component gọi hàm đọc tham số đầu vào **trong hàm khởi tạo**, trước khi khung ứng dụng gán tham số	Chuyển sang phương thức chạy sau khi khởi tạo
8	Bộ nạp dữ liệu mẫu cộng dồn mỗi lần khởi động	Điều kiện chống trùng khoá theo ngày tương đối nên mỗi ngày lại khớp khác	Đổi điều kiện sang khoá toàn bảng
9	Khu quản trị đóng nhầm phòng không hiển thị	Khối giao diện giữ định danh phòng cũ sau khi bộ lọc của màn hình cha thay đổi	Đối chiếu định danh với danh sách phòng đang hiển thị
10	**Danh sách đơn nói "bấm vào mã đơn để mở chi tiết" nhưng không bấm được**	Ô mã đơn là ô bảng thường; chỉ một liên kết ví dụ dưới bảng mở được đơn **đầu tiên**, mười chín đơn còn lại không có lối vào	Thêm kiểu cột liên kết cho bảng dữ liệu dùng chung, mã đơn của mọi hàng thành thẻ liên kết thật
11	Ô chọn tệp ảnh không có tên cho trình đọc màn hình	Ô ẩn bằng lớp chỉ-đọc-màn-hình nhưng **vẫn nằm trong thứ tự Tab**, thành một điểm dừng không tên	Bỏ ô khỏi thứ tự Tab và đặt nhãn cho nó; nút bấm nhìn thấy được mới là lối vào